

电子科技大学

UNIVERSITY OF ELECTRONIC SCIENCE AND TECHNOLOGY OF CHINA

学士学位论文

BACHELOR THESIS



论文题目 小型旋翼机路径规划算法的研究与应用

专 业 自动化

学 号 2013070906006

作者姓名 张泽

指导教师 李瑞 副教授

摘 要

随着小型旋翼无人机的相关技术理论日益成熟，由于其稳定性高、成本低、功能性强等优点使之在各个领域均有广泛应用。路径规划一直是实现飞行器自主飞行的重要支撑技术，是提高飞行器智能程度的关键课题，其根本目的是在一定限制条件下得到所给代价最小或充分小的飞行路径。

本文将针对小型四旋翼无人机进行实际路径规划过程算法的研究分析和比较，在通用路径算法的基础上进行修正和拓展。最终将通过分析以典型动态模型和控制算法生成的四旋翼无人机模型，得到一定的可行性分析结果：以本文提出的规划程序作为理论依托可以有效且可行的达成多旋翼无人机的自主飞行任务。

关键词：小型四旋翼无人机，路径规划，Voronoi 图，Dijkstra 最短路，A*算法

ABSTRACT

With the gradual maturation of relevant theories about Mini Multi-rotor Unmanned Aerial Vehicle (abbr. Mini MRUAV), Mini MRUAV has been applied extensively to various fields due to its advantages of high stability, low cost and strong functions. Path planning is always a significant technique which supports the realization of UAV's purposeful autonomous flight. It is one of the key issues in improvement of UAV's intelligence. The objective of path planning is to obtain the flight path which could achieve the least cost and danger, based on the specific restrictive condition.

A practical path planning procedure is analyzed and compared with other methods in this dissertation. The common path planning methods are improved and expended for MRUAVs. In the end of this dissertation, real-time emulation is implemented with a quadrotor model which is generated by a simplified dynamic model and typical control algorithm. From the result of emulation, it is effective to fulfill MRUAV's autonomous navigation based on planning procedure proposed in this dissertation.

Keywords: Mini Multi-rotor Unmanned Aerial Vehicle, Practical Path Planning, Voronoi Diagram, Dijkstra algorithm, A* algorithm

- 目 录 -

第一章 绪 论	1
1.1 研究背景及现状	1
1.1.1 四旋翼无人机的研究背景及现状	1
1.1.2 路径规划的研究背景及现状	3
1.1.3 飞行器路径规划的研究背景及现状	3
1.2 研究内容及章节安排	4
1.2.1 研究内容	4
1.2.2 章节安排	4
1.3 本章小结	5
第二章 理论研究	6
2.1 路径规划概述	6
2.1.1 路径规划的定义	6
2.1.2 路径规划的任务	7
2.1.3 路径规划的相关算法	7
2.2 沃罗诺伊图 (Voronoi diagram)	8
2.2.1 沃罗诺伊图的概念	8
2.2.2 沃罗诺伊图的生成和特性	9
2.3 最优路径规划	10
2.3.1 迪杰斯特拉算法简介	10
2.3.2 迪杰斯特拉算法实现	10
2.3.3 A*算法简介	11
2.3.4 A*算法的实现	11
2.3.5 杜宾斯曲线简介	12
2.3.6 杜宾斯曲线的生成	13
2.4 辛格莫伊德函数 (Sigmoid function)	14
2.4.1 辛格莫伊德函数简介	14
2.4.2 辛格莫伊德函数扩展	14
2.5 本章小结	15
第三章 模型搭建	16
3.1 四旋翼飞行器建模	16

3.1.1 欧拉角及姿态解算	16
3.1.2 四旋翼飞行器动力学模型	18
3.2 控制算法	20
3.2.1 位置控制	20
3.2.2 姿态控制	22
3.2.3 Sigmoid 控制器.....	22
3.3 模型控制	24
3.4 本章小结	24
第四章 设计应用	25
4.1 路径规划条件分析	25
4.1.1 威胁点的性质及筛选	25
4.1.2 算法匹配	26
4.2 基于沃罗诺伊图的飞行路径规划	27
4.2.1 沃罗诺伊图的生成	27
4.2.2 沃罗诺伊图的使用	28
4.2.3 A*算法实现障碍物规避	29
4.2.4 迪杰斯特拉算法实现雷达规避	30
4.2.5 飞行轨迹设定	32
4.3 基于杜宾斯曲线的转向策略	33
4.4 本章小结	36
第五章 分析和总结	37
致 谢	40
参考文献	41
外文资料原文	43
外文资料译文	44

第一章 绪 论

1.1 研究背景及现状

多旋翼无人飞行器（Multi Rotors Unmanned Aerial Vehicle, abbr. MRUAV）是一种具有三个或三个以上数量的水平旋翼的无人驾驶直升机。四旋翼无人机的结构简图如图 1-1 所示。

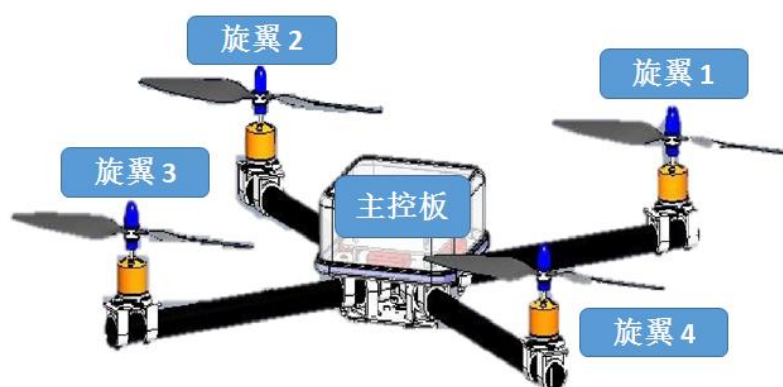


图 1-1 四旋翼无人机结构简图

随着控制理论的发展、电子和计算机技术的进步，微型多旋翼无人机由于其体积小、结构简单、成本低廉、易维护和维修、可悬停以及可低速航行、可拓展性强即可搭载多种传感器等优点和特点，被广泛用于科学研究、商业消费和军事装备，除了作为普通消费品供娱乐和航拍，无论是在地形勘察、人员搜救、物资运送、高空作业等专业领域，还是敌情侦查、区域巡逻、火力掩护、火力打击等军事领域，多旋翼无人飞行器都有着独特的优势以及快速的发展^[1-5]。

路径规划是广泛应用于机器人智能化过程的一种工具，其主要目的是在规定的条件下找到代价最小或收益最大的安全路径。对于飞行器来说，其路径规划相比地面机器人更为复杂，需要考虑高度和更多姿态问题，因为通用的路径规划算法生成的路径可能并不适用与飞行器。显而易见的一点是，实现飞行器的自主飞行对飞行器的功能拓展是很关键的技术，所以针对飞行器的路径规划算法的研究一直是热点问题^[6-7]。

1.1.1 四旋翼无人机的研究背景及现状

1907 年由法国人 Breguet 兄弟实现的 Breguet-Richet 四旋翼载人飞行器是有记载的最早的四旋翼飞行器^[8]（如图 1-2 所示），该飞行器仅使用油门控制而没有任

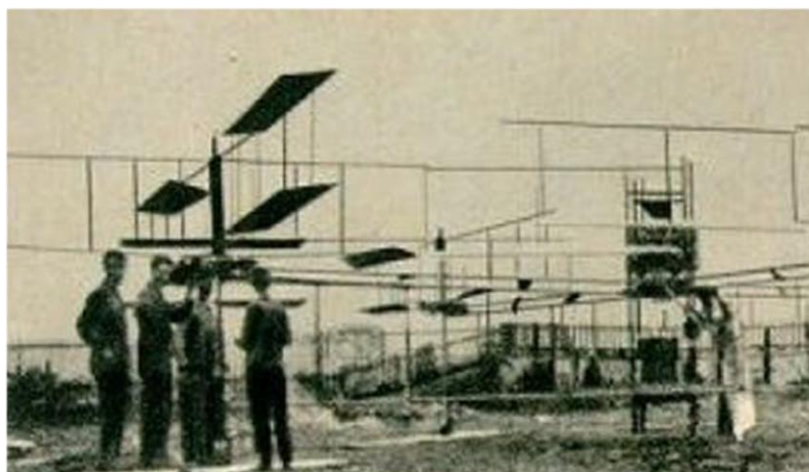


图 1-2 Breguet-Richet 四旋翼载人飞行器

何辅助控制设备；1921 年，美国空军与 George de Bothezat 和 Ivan Jerome 关于研发垂直起降飞行器而进行了合作^[8]，在大约两年的时间里，该四旋翼飞行器共进行了 100 次左右的试飞，虽然限于历史条件没能达到实用水平，但是这种飞行器对后续理论和应用都有重大意义。在随后的几十年间，不断有人或团队尝试改进四旋翼飞行器技术，不过直到近年来相关技术水平的提高，尤其是控制理论、电机技术、传感器技术、嵌入式系统等领域的发展，四旋翼飞行器的研究价值和实用性能才得以产生了质的飞跃。

在高校和科研机构对四旋翼飞行器的研究过程中，有不少突破性的令人印象深刻的成果。比如美国宾夕法尼亚大学基于 HMX-4 模型研制的四旋翼飞行器以及定位系统可以提高飞行器飞行的稳定性，并可以完成定位、交互等任务；TED Global 机器人实验室的 Raffaello D'Andrea 团队曾演示过一个具有惊人运动性能的基于室内定位系统的四旋翼飞行器；瑞士联邦理工学院使用不同的控制算法——PID 算法和 LQ 算法——来控制飞行器，并通过特定的测试平台对拟飞行表现进行评估，实验表明 PID 算法更加有效。此外，诸如美国斯坦福大学的 STARMAC 项目、麻省理工大学的飞行器协同项目等都是在四旋翼飞行器领域较具代表性的。相比之下，



图 1-3 DJI PHANTOM 4 PRO 产品展示

在商业开发的过程中，四旋翼飞行器的更新速度似乎要快的多，其中较为知名的有 RC Toys 公司的 Draganflyer 系列、法国 Parrot 公司的产品、美国 3D Robotics 公司的 Solo 航拍机、中国大疆无人机（如图 1-3 为大疆精灵系列产品）等。其中大疆无人机在全球的市场份额遥遥领先于其他公司。

1.1.2 路径规划的研究背景及现状

路径规划是运动规划的主要研究内容之一，其和轨迹规划一同构成了运动规划的主要问题。路径规划的研究内容可以概括为寻找连接起点和终点的策略，通常是针对位置而言。根据运动物体的感知能力和环境的感知度可以对路径规划进行三种划分：全知环境下的静态规划、未知环境下的动态规划以及限定环境下的动态规划。目前有几种比较流行的规划算法：在全知环境下的空间分割栅格法、简化拓扑法、概率路径法、空间构图法等，以及限定或未知环境下的人工智能领域的算法。以传感器技术的发展为基础，以空间遥感技术为辅助，随着智能化要求的提高，路径规划在众多科技研究和应用上都十分重要^[9]。

通用的路径规划算法可以分为两大部分，首先生成部分或所有可用路径，或称备选路径，然后从备选路径中计算最优路径。空间分析中的泰森多边形分析，数值方法中的曲线插值方法等可以在不同程度上生成可用路径；概率统计学方法中的模拟退火法，图论中的迪杰斯特拉算法，人工智能技术中的 A*算法、蚁群算法、神经网络模型等都可以用于计算一定限制条件且不同代价下的最优路径。这些常用的路径规划算法最终得到的是基于路径节点的路径信息，属于指导性结果，并没有考虑将要使用这些信息的受体，一般来说对于实际的受体，可能需要对常规路径规划的结果进行修正。

1.1.3 飞行器路径规划的研究背景及现状

较之地面单位的路径规划，飞行器由于其脱离地面，当考虑飞行器的路径规划时，需要对一些影响因素进行转变。如将地形坡度转变为距地高度，将左右转向转变为姿态解算等；由于很多飞行器如固定翼飞行器由最小速度的限制，在路径规划时往往还要将最小速度考虑在内。可以看到，飞行器的路径规划需要结合飞行器的特点，实际的路径规划问题不仅是通用的路径规划算法，还涉及高度获取、威胁评估与处理、轨迹规划等技术。

路径规划长久以来都是需要人力参与实现的，越来越复杂的路径规划需求促使人们开发自动路径规划技术，上世纪 80 年代左右出现了自动路径规划的概念，此前一直由人类担当路径规划中的处理单元。1984 年美国空军莱特航空实验室提

出了“战术飞行管理”的概念，这一概念的核心技术就是最优路径规划。时至今日，国际国内都对这一问题做了大量的研究和应用，主要是针对有人驾驶飞行器的辅助飞行管理、无人机巡航、飞行器的编队飞行、小型旋翼机的室内飞行等。其中不仅包括国内外知名高校研究所等科研机构，以及各国航空航天局、空军相关部门等政府机构，也包括以上提到过的众多商业组织和机构。飞行器路径规划正是飞行器相关研究中亟待发展的部分。如图 1-4 是拉斐尔·安德烈(Raffaello D'Andrea)在 TED 演讲中展示旋翼机协作。



图 1-4 拉斐尔·安德烈在展示四旋翼的协作

1.2 研究内容及章节安排

1.2.1 研究内容

本次课题的主要目的是对多种通用的路径规划算法通过组合、改进等手段，并考虑四旋翼无人机在飞行过程中的一些特性，包括环境特性、物理学特性和控制过程等，建立完整的四旋翼无人机飞行路径规划。

本次研究的主要目标是飞行路径规划，为此将对飞行区域的环境进行设置，并针对不同环境的特点对应的采用不同的路径规划算法得到所需飞行路径，还使用了可选的路径优化方法；除此了主要目标之外，还提出了一种控制算法，结合四旋翼无人机的特点和模型对无人机在路径规划算法生成的路径上飞行的过程进行规划，从而完成了四旋翼无人机整体的运动规划。

1.2.2 章节安排

全文的主要部分将按照路径规划算法理论研究、四旋翼无人机模型建立、路径规划的实际构想和实现以及分析和总结四个部分来依次进行。

具体安排如下：

第一章为绪论，将对研究背景和发展做出介绍，并概述课题研究内容；

第二章为理论介绍，将对课题中使用的理论背景做出介绍和一定的说明；

第三段为模型搭建，将对本次课题使用的四旋翼无人机仿真模型和相关控制算法进行简单的推导和说明；

第四段为设计应用，将对本次课题的设计及实验主体进行介绍和详细说明；

第五段为分析总计，将对课题结果进行分析和并对整个设计过程进行总结。

1.3 本章小结

本章主要介绍了本次课题——四旋翼无人机的路径规划的理论背景和实际意义，简要展示了四旋翼无人机的历史发展和研究现状，结合路径规划的含义和在本文中的拓展，概括性的说明了该课题的主要研究内容和研究意义。

第二章 理论研究

2.1 路径规划概述

通常意义上，路径规划是在限定条件下的对质点模型进行的，旨在找到最优或较优的路径使得起始点和目标点之间得以连通，对这种路径的寻找策略就是路径规划。如果我们把质点模型替换为其他模型，如四旋翼飞行器模型，并把新模型的特性也设置为条件，一般会对路径规划的过程产生一些影响。比如对固定翼无人机来说，有最小速度的限制，也就是固定翼无人机的速度无法小于某一值，因此规划的路径的曲率会受到限制，也不能出现折线这种情况。而四旋翼无人机可以实现悬停，因此理论上路径可以存在折线。联系四旋翼无人机的特性对飞行路径规划算法进行验证是本次课题的研究目标之一。

2.1.1 路径规划的定义

路径规划是指在规定的或已知的条件下，依据特定的评价标准，寻找一条从起始点或状态到达目标点或状态的合理路径。在该定义下，隐含了几点路径规划的要求和特点。首先，路径规划一定是在限定条件下进行的，比如对机器人的避障路径规划中，避开障碍物就是规定的条件，如果障碍物的位置是已知的，则其就可作为已知的条件；其次，一定有一个评价标准来指导路径规划策略目标，并对规划的路径进行评判，常用的评价标准有路径长短、安全系数、路径权值最高或最低等；最后，合理的路径是指路径的规划需要有一定的允许范围，比如对路径长度作出限制，或对路径所在区域作出限制，只有在限制之内才被视为合理，否则视为不合理即不可被接受的路径。

通常以四个步骤进行路径规划：规划空间建模、路径生成、路径选择、路径优化。

一般通用的路径规划研究是以质点模型为运动物体的，其重点在于路径本身，而不在于将要遵循该路径运动的物体；但是如果将模型替换为更复杂的物体，比如四旋翼飞行器模型，并将新模型的一些我们关心的特性也作为路径规划的条件，也许会对路径规划策略产生影响，并且需要将路径规划拓展到运动规划上。在本次课题中，认为路径规划可以向轨迹规划去拓展，从而考虑基于路径规划的运动规划。

2.1.2 路径规划的任务

路径规划的任务可以简单的归结为是找到从起点 A 到终点 B 的路径。理想条件下, 从 A 到 B 的路径总为直线, 但是往往存在一些限制因素使得路径无法为直线, 这也是路径规划主要解决的问题, 即在一定限制条件下寻找 A 到 B 的路径。

正如前面所提到的, 不同的环境下, 路径规划的任务有所不同。在未知环境或动态环境下, 路径规划更多的是实时的, 即在充分短的时间内处理出现在原先路径上的限制因素并生成新的行进方向和路径。比如在障碍物未知的情况下让机载探测传感器的机器人穿越一片有若干障碍物的地区, 此时就需要当探测器探测到障碍物后, 路径能够被重新评估和规划以便避开障碍物。而在全知环境下, 路径规划算法虽然不需要着重处理突发状况, 但是往往会存在更加严格和丰富的限制, 使得规划的难度变大。事实上, 绝对的全知环境是理想化的, 也是不存在的。

除了环境限制, 算法的受体也会对规划有所影响, 主要是其性能和状态的限制, 比如前进速度、加速度、行进方向、碰撞体积, 还有针对飞行器的飞行高度等问题。除了前面提到的固定翼飞行器有最小飞行速度的例子, 还有一个比较典型的例子是前进方向, 对于无法原地改变前进方向或限制改变前进方向的受体, 就要考虑出发和到达曲率的问题。

综合考虑环境和受体的种种因素, 才能获得具有切实可行性的路径规划算法, 也是路径规划真正的任务。

2.1.3 路径规划的相关算法

路径规划算法的种类很多, 这得益于该问题长久以来的发展和研究者们持续的关注。根据研究领域和研究方向, 可以将这些方法大致分为以下几类^[9]。

传统算法是指出现较早, 以传统直观思想为核心的算法。传统的算法有模拟退火法、人工势场法、模糊逻辑法、禁忌搜索法, 这些方法都是基于对自然现象或人类行为的模拟, 因此可以直观的去理解这些算法。这些算法需要产生一个用于模拟规划过程的规则, 以函数或表格的形式表现出来。

传统算法的直观性也正是其局限性, 随着模型复杂度的提高, 传统算法往往难以简洁快速的建立环境模型。为了从整体的层面出发建立路径空间模型, 人们引入图形学算法用于建立路径规划的空间模型。比较常用的有 C 空间法、自由空间法、栅格法、Voronoi 图法等, 这些方法都是通过建立局部或整个路径空间 (或称地图) 将路径规划中限定的条件具化或描述出来。

虽然图形学算法可以建立整个地图, 但是往往没有搜索路径的能力, 需要结合其他的方法算出最终的路径。一些经典算法常用来配合图形学算法, 比如 A*算

法、Dijkstra 算法、Floyd 算法等，这些算法各自对处理某些类型的路径规划问题十分有效。

随着人工智能的发展，人们开始使用更加智能化的学习或启发式算法，包括在路径规划问题上，一般将这些算法称为智能仿生算法，比较有代表性的有蚁群算法、神经网络算法、遗传算法、粒群算法等。

路径规划的算法繁多而适应情况各有不同，本次课题选用多种方法，包括 Voronoi 图、Dijkstra 算法、A*算法、蚁群算法、Dubins 曲线进行路径规划，并提出了一种基于 Sigmoid 函数的位置控制器，结合四旋翼飞行器的特点分析尝试验证相关飞行路径规划算法的可行性。

2.2 沃罗诺伊图 (Voronoi diagram)

2.2.1 沃罗诺伊图的概念

Voronoi 图（中文翻译为沃罗诺伊图或维诺图）是计算几何学中一种基础数据结构，1975 年一篇关于邻近点问题的论文在 IEEE 上发表，其中对 Voronoi 图的讨论直接导致了计算几何学的诞生，可见 Voronoi 图的重要地位。其在二维空间简要的数学定义如下^[10-11]：

设 S 为平面 P 内一点集，其中共有 n 个不同的元素。首先定义对两个不同的点 s_1 和 s_2 ，且 $s_1, s_2 \in S$ ， s_1 对 s_2 的近邻区域是指，和 s_2 相比距离 s_1 更近或同样近的区域 P_{s_1, s_2} ，且 $P_{s_1, s_2} \in P$ ，也称 s_1 对 s_2 的支配，表示为

$$dom(s_1, s_2) = \{x \in R^2 | \delta(x, s_1) \leq \delta(x, s_2)\} \quad (2-1)$$

其中 δ 表示欧氏距离。

在式 (2-1) 定义的基础上可以于 P 中划出 n 个区域，第 i 个区域 P_i 满足

$$P_i = \bigcap_{j=1}^n dom(s_i, s_j) \quad (2-2)$$

其中 i 是不大于 n 的任意自然数。由平面 P 、点集 S 和区域 P_i 组成的几何图形就是 Voronoi 图。

直观的理解是，Voronoi 图可将存在一系列给定点的平面划分成这样的一些区域或称胞元 (cell)，每个胞元只包含一个给定点，且胞元内到被包含的给定点的距离小于其他任何一个胞元外的给定点。

Voronoi 图问题又被称为或一定程度上等价于狄利克雷镶嵌问题 (Dirichlet tessellation)、泰森多边形问题 (Thiessen polygon) 或德劳内三角剖分问题 (Delaunay

triangulation), 其目的就是找到点集的分界线, 以将整个区域以点为中心进行划分, 使得划分后的图形具有上文中所说的特点。

如图 2-1 是针对在 0 到 1 之间随机生成的一系列点生成的 Voronoi 图。

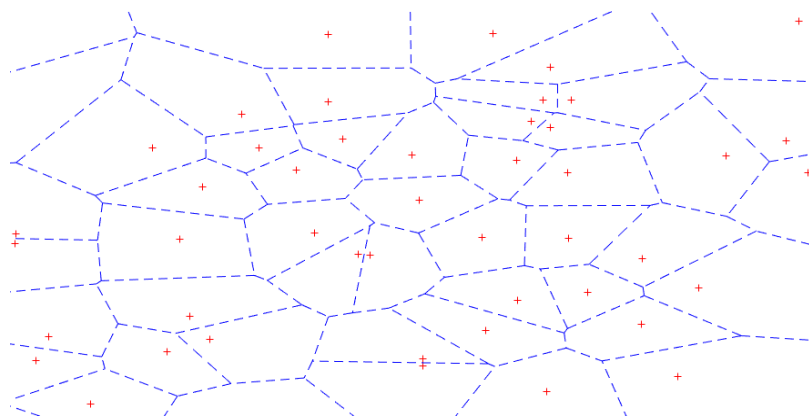


图 2-1 Voronoi 图示例

2.2.2 沃罗诺伊图的生成和特性

普通 Voronoi 图的生成方法一般分为两类, 一类是直接法, 即直接生成 Voronoi 图, 包括定义法、增量法、分治法、扫描法等; 另一类是间接法, 即依靠 Delaunay/德劳内三角剖分 (事实上, Delaunay 三角剖分图和 Voronoi 图有对偶关系), 通过换边法、升维法等构造 Voronoi 图^[11]。本次课题使用 MATLAB 封装函数中的 Voronoi 函数, 该函数使用 Delaunay 三角剖分, 再找到剖分而成的三角形外心, 将相邻外心连接起来即得到 Voronoi 图。具体的算法过程中, 需要建立表格, 判断相邻点、相邻边等, 在后面会大致进行解释。

根据 Voronoi 图的定义及生成过程可以看出, 得到的胞元边界线是对应区域内距离所有点最远的轨迹, 正是这个特点, 使得 Voronoi 图可以用于路径规划当中来。并且 Voronoi 图是针对定点信息的, 能够简洁的建立直观的数学模型。但是其缺点也十分明显, 首先是对于不同的点集, 生成的图像也是不同的, 增删点对图像必然会产生一点的影响, 而在点集庞大的时候, 每产生一次 Voronoi 图就会花费较长的时间。也因此原始的 Voronoi 图不适用于动态点产生的区域划分问题。然而, 通过一些规则的制定和算法的改进, 可以在一定程度上解决这种问题。其次, Voronoi 图生成的路径是距离给定点最远的路径, 即便在有的场合, 比如下面介绍规避雷达监测的情况, 这种距离给定点最远的路径规划是有用的并且必要的, 不过这也势必会浪费很大的空间, 这一点在下面具体介绍规避障碍物的时候会显现出来。最后, Voronoi 图无法单独应对复杂的地理环境, 不过在本文中不会涉及到过于复杂的飞行环境。

2.3 最优路径规划

2.3.1 迪杰斯特拉算法简介

Dijkstra 算法(中文翻译为迪杰斯特拉算法)是荷兰计算机科学家埃德斯加·狄克斯特拉在 20 世纪 50 年代提出的一种以图论为理论基础的最短路径算法,解决的是有向图中从单源点向其他所有节点的最短路径的求解。其主要特点是遍历性,即遍历图中所有的点。需要注意的是,该算法要求路径权值不能为负。

2.3.2 迪杰斯特拉算法实现

迪杰斯特拉算法的实现可以用图论的形式表示: 设 $G = (V, E)$ 是一个有向图, 将顶点集合 V 分为两个活动的子集 V_1 和 V_2 , 前者表示已经求解过的节点集合, 后者表示还未求解的节点集合, 初始状态下, 仅有源节点在 V_1 中, 其余点均在 V_2 中, 通过一个原则将 V_2 中的点依次移到 V_1 中, 直到所有节点都被遍历过为止, 该原则描述性的表示是在以当前点为准修改了所有路径长度后, 从当前点到下一点的带权路径长度最短^[12]。算法的流程图如图 2-2 所示。

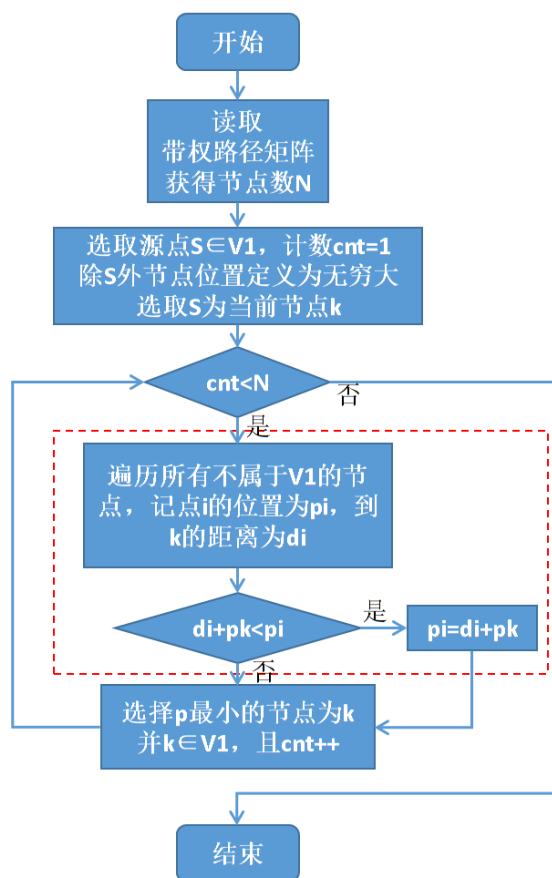


图 2-2 Dijkstra 算法流程图

2.3.3 A*算法简介

A*算法是解决最短路径规划问题的一种启发式搜索算法，于上个世纪 60 年代末被斯坦福研究所的 Peter E. Hart、Nils J. Nilsson 和 Bertram Raphael 提出并发表在 IEEE 期刊^[13]。所谓启发式搜索(heuristic search)，简单来说是指利用经验知识加快搜索速度的搜索方式，A*算法通过建立估价函数来加入经验知识从而加快搜索速度，相比于深度优先搜索、广度优先搜索，A*算法扩展的节点数往往少很多。

A*算法的核心是建立估价函数，其结构为

$$f(n) = g(n) + h(n) \quad (2-3)$$

其中 $f(n)$ 为估价函数，用于估计节点的代价； $g(n)$ 为代价函数，是指起点到当前节点的实际最小代价，或称最短路径长度； $h(n)$ 为启发式函数，是当前节点到终点的预估最小代价。可以看到，启发式函数是由经验知识得来的，其估计的值对 A*算法的结果有决定性影响。另外，Peter E. Hart 等人还得到了保证所得路径是最短路径的条件，称为可接纳条件：

$$h(n) \leq h^*(n) \quad (2-4)$$

即估计代价不大于实际代价，或称估计的路径长度不大于实际的路径长度。

2.3.4 A*算法的实现

正如上文所说，A*算法的核心在于建立估价函数，在实际的使用中，配合 OPEN 表和 CLOSE 表，通过估价函数对节点的扩展方向加以确定，在满足估计代价不大于实际代价的情况下，可以保证所得路径最短^[13-14]。A*算法的流程如图 2-3 所示。

整个算法可以分为四个步骤：初始化，目标判别，节点扩展和估价排序。在进一步分析算法流程之前，需要对一些概念做出解释：OPEN 表用以储存扩展出来的待处理的节点，CLOSE 表用以储存处理过的节点，流程图中的 N 点又称为当前节点。

一、初始化

初始化就是将起点 S 放入 OPEN 表，表示从该点开始扩展。

二、目标判别

初始化后进入循环过程，首先是判断 OPEN 表是否为空，以检查算法是否失败，对于封闭的图来说是不会存在失败的情况的。然后将 OPEN 表中首个节点作为当前节点并检查该节点是否为终点。以上两步称为目标判别。

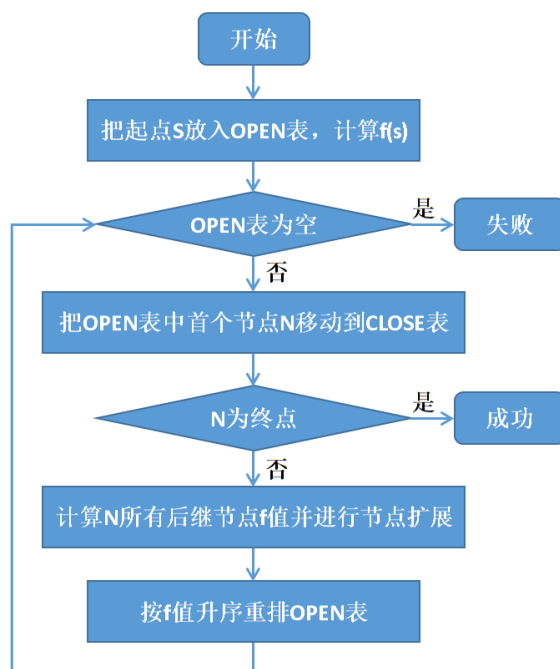


图 2-3 A*算法流程图

三、节点扩展

接下来扩展当前节点，是最重要的一步，扩展节点的原则是：①若某后继节点不在两表中，将该后继节点加入 OPEN 表并将其指针指向当前节点 N，指针用于回溯路径；②若该后继节点在 OPEN 表，且其新 f 值小于原 f 值，更新其 f 值并将其指针指向当前节点 N；③若该后继节点位于 CLOSE 表，且其新 f 值小于原 f 值，则移回 OPEN 表，更新 f 值并将其指针指向当前节点 N。关于估价函数的建立将在第四章讨论，这里不再赘述。

四、估价排序

最后按照估价函数的函数值，以升序重新排列 OPEN 表，使得表中首个节点是估价最小的节点。然后回到第二步，循环往复直至到达终点。到达终点之后，根据节点扩展这一步中建立的指针信息，可以回溯找到从起点到终点的最短路径。

2.3.5 杜宾斯曲线简介

Dubins 曲线^[15]（直译为杜宾斯曲线）最早由 L. E. Dubins 于 1957 年提出，旨在寻找满足一定曲率条件下，连接同平面具有特定方向的任意两点的最短轨迹。一般同一平面上的任意两点的最短路径是直线，路径长度即直线距离，而在一定的曲率的情况下，也就是有一定的出发方向和到达方向，最短的路径则是圆弧。

Dubins 曲线路径可被简单地定义为，在最大曲率限制下，平面内两个有方向的点之间的最短可行路径是 CLC 路径或 CCC 路径，或是它们的子集，其中 C 表示圆弧段，L 表示与 C 相切的直线段，本文只讨论 CLC 路径。

Dubins 曲线在机器人的路径规划中应用广泛，在无人机领域，事实上 Dubins 曲线已经十分接近于真实的飞行轨迹，尤其是和常规路径规划所生成的直线、折线路径相比。虽然在之前讨论四旋翼飞行器的优点时提到，四旋翼飞行器具有空中悬停和定点转向的能力，但是由于大部分小型四旋翼飞行器使用锂电池等电源供电，而受限于电池的性能，四旋翼飞行器的滞空时间有限，相比于悬停转向，曲线转向对电源的消耗更少，所以应该根据实际条件调整四旋翼飞行器的转向模式。因此本文将讨论 Dubins 曲线作为四旋翼飞行器转弯模式的扩展。

2.3.6 杜宾斯曲线的生成

为了简化计算，假设初始点和终止点的距离足够远，也就是说保证规划的路径由圆弧、直线、圆弧三段组成；容易想到，如果两点距离比较近，则有可能出现规划的路径由两段圆弧和一段直线一段圆弧组成的情况，如果两点相距过于接近，则有可能导致无法求出 Dubins 路径，对于这些初始点和终止点相距较近的情况，将不做讨论。具体的限定条件为，初始点和终止点之间的距离不小于 $2/C_{max}$ ，其中 C_{max} 是转弯最大曲率^[3]。

设出发点为 P_s ，出发航向为 φ_s ，终止点为 P_e ，终止航向为 φ_e ，如图 2-4 所示，以航向为标向，航向所在直线为切线，以 r_c 为半径，其中 r_c 为最大曲率的倒数，即 $r_c = 1/C_{max}$ ，依附出发点和终止点都可以绘制一个左圆（逆时针圆）和一个右圆（顺时针圆）。图 2-4 展现了 Dubins 曲线的生成过程。

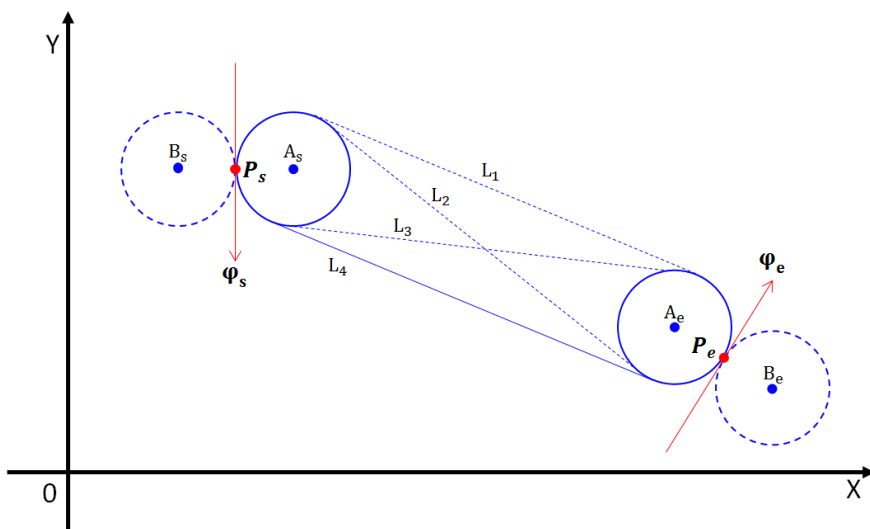


图 2-4 Dubins 曲线的生成

2.4 辛格莫伊德函数 (Sigmoid function)

2.4.1 辛格莫伊德函数简介

Sigmoid 函数 (中文直译为辛格莫伊德函数), 又称 S 型生长曲线, 是神经网络中使用十分广泛的一种激励函数^[16], 其基本表达式为

$$S(x) = \frac{1}{1+e^{-x}} \quad (2-5)$$

可以看到该函数连续可导, 且其导函数也连续可导; 并且 Sigmoid 函数单调递增, 具有平滑性和饱和性。其导函数表达式为

$$\dot{S}(x) = S(x) - S^2(x) = \frac{e^{-x}}{(1+e^{-x})^2} \quad (2-6)$$

事实上, 由式 (2-6) 的导函数形式可以看出 Sigmoid 函数具有无限可导的特点。

如图 2-5 是 Sigmoid 函数及其相关导函数的函数图像。

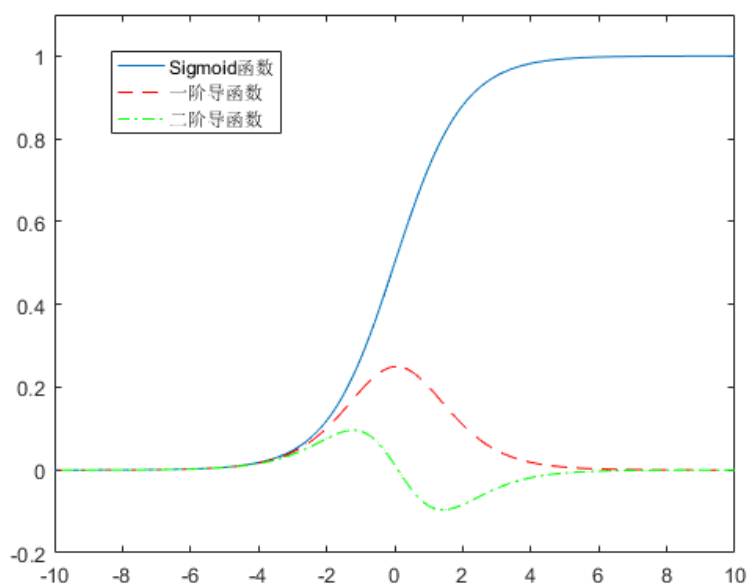


图 2-5 Sigmoid 函数及其相关导函数

2.4.2 辛格莫伊德函数扩展

对 Sigmoid 函数进行修正可以得到一个一般形式的修正 Sigmoid 函数,

$$S_c(x) = \frac{K}{1+e^{-a(x-x_0)}} + d \quad (2-7)$$

其导函数为

$$\dot{S}_c(x) = \frac{aKe^{-a(x-x_0)}}{(1+e^{-a(x-x_0)})^2} \quad (2-8)$$

在式 (2-7)、(2-8) 中, K 表示增益, a 表示陡度, x_0 表示横向偏移量, d 表示纵向偏移量。其意义是, K 表示函数值的放大倍数, K 越大幅值越大; a 表示函数值的变化速度, 或称陡度, a 越大则函数值变化越快而变化区间越小; x_0 表示函数沿 X 轴的平移量, 也是函数值变化最快的即导函数最大点的自变量值; d 表示函数沿 Y 轴的平移量。

本次课题引入了依照 Sigmoid 函数为位置曲线设计的控制器辅助仿真, 会在下一章具体说明。

2.5 本章小结

本章主要对本次课题涉及的理论知识, 包括路径规划、Voronoi 图、Dijkstra 算法、A*算法、Dubins 曲线和 Sigmoid 函数进行了定义性或描述性的介绍。这些理论背景是本次课题设计及仿真的知识储备和保证。

第三章 模型搭建

3.1 四旋翼飞行器建模

欠驱动系统是指系统的独立控制变量少于系统自由度个数的一类系统，四旋翼飞行器是一种四通道输入、六通道输出的欠驱动动力学旋翼直升机，其原理和结构相对简单，执行效率高且十分灵活。本次课题虽然不是以建立四旋翼飞行器的模型为主要内容，但是为了联系四旋翼飞行器的一些特性来设计或者验证飞行路径规划的有效性，需要对四旋翼飞行器的一些基本规律进行了解。

四旋翼飞行器以十字形机架为主要机械结构，四个端点为四个电机，相对的电机为一组，两组电机反向旋转以平衡飞行时的陀螺效应和空气动力扭矩效应。机架中心为飞行控制计算机，为核心控制部分。其余部件包括电子调速器用以驱动电机、无线传输器（或集成在飞行控制板中）用以通信等等。

控制程序的步骤一般是先由惯性测量单元 IMU(Inertial Measurement Unit)测量三轴加速度和三轴角速度，然后单片机进行基于某种算法（如四元数、方向余弦等）进行姿态解算，求解出俯仰/翻滚/偏航（Pitch/Roll/Yaw）三个欧拉角角度值，通过控制器（如 PID 控制）运算后输出四路 PWM 波控制四个电机。

3.1.1 欧拉角及姿态解算

地球是常用的参考坐标系，严格的地球坐标系定义是原点位于地球中心，坐标轴与地球固连，轴向定义为 Ox 、 Oy 、 Oz 三个轴。其中， Oz 沿地球极轴方向指向北极， Ox 轴沿格林尼治子午面和地球赤道平面的交线指向格林尼治子午线， Oy 和 ZOX 构成右手坐标系。地球坐标系绕 Oz 轴以自转角速度转动。

随体坐标系是指原点位于飞行器重心，三条正交轴线 ox 、 oy 、 oz 分别指向飞行器正向、右侧和下方的坐标系。

由于四旋翼飞行器在飞行过程中会有姿态的改变，也就是会在地球坐标系 E 系上产生和 $O-XYZ$ 三个轴的夹角，为了便于分析，需要找到一种方法，使物理量能够在 E 系和跟随飞行器的随体坐标系 B 系之间进行转换，这个转换的过程就称为姿态解算。为了方便表示， B 系的坐标轴用 $o-xyz$ 表示

最直观的表达是 B 系和 E 系的坐标轴或坐标平面间的夹角，但是由于无人机的运动是以旋转的形式，用夹角不仅难以表现动态的变化，更难以用来作为控制的参考量。因此引入欧拉角的概念。

欧拉角是用以描述刚体围绕定点旋转的独立角参量，基本定义是由章动角 θ 、进动角 ψ 和自转角 ϕ 组成，分别表示 Oz 到 oz 轴的旋转角、 Ox 到垂直于 OZz 平

面的节线 ON 的旋转角、ON 到 ox 的旋转角，如图 3-1 就是对欧拉角的描述。实

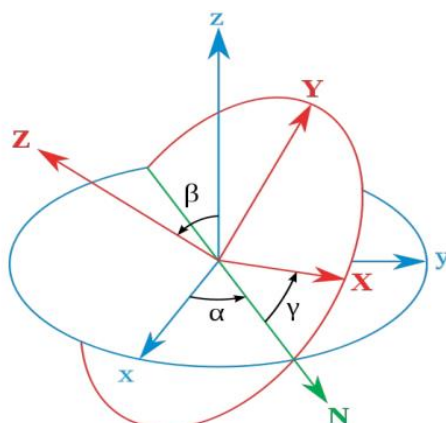


图 3-1 欧拉角的定义 (ZXZ 序)

际上可以进行一定的拓展，为了方便理解和计算，本文将按照如下的定义：E 系统 X 轴旋转 ϕ ，再绕经 X 轴旋转过的坐标系的 Y' 轴旋转 θ ，最后绕经过两次旋转的 Z'' 轴旋转 ψ 得到 B 系。将这三次旋转对应的角度分别翻滚角、俯仰角和偏航角，对应的操作就是翻滚、俯仰和偏航 (Roll、Pitch、Yaw)。以下将用欧拉角来统一称呼这三个角。图 3-2 展示了飞行器中欧拉角的直观形式。

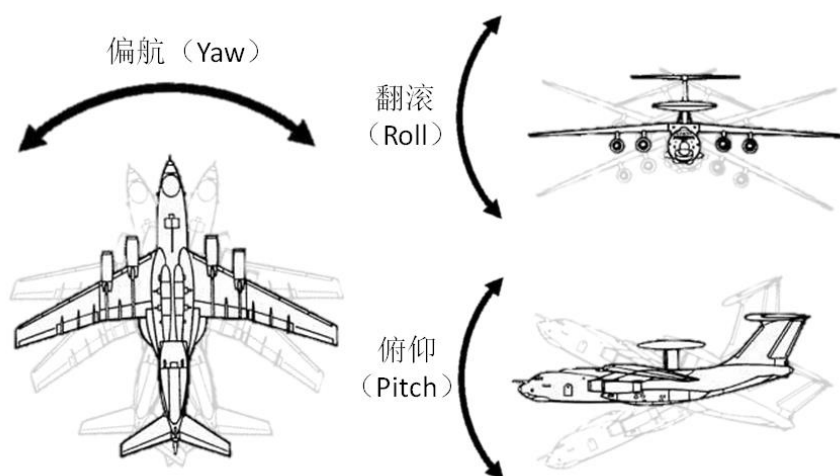


图 3-2 固定翼飞行器姿态角演示

为了建立欧拉角的数学描述，我们首先引入二维笛卡尔坐标系 (2-D) 中的向量旋转公式：设 2-D 中向量 $\vec{P}$ 的坐标为 (x_0, y_0) ，绕坐标原点逆时针旋转 α 角后（或顺时针旋转 $-\alpha$ 角），得到的新向量 $\vec{P}_1$ 坐标为 (x_1, y_1) ，则坐标值有如下关系：

$$\begin{cases} x_1 = x_0 \cdot \cos \alpha - y_0 \cdot \sin \alpha \\ y_1 = x_0 \cdot \sin \alpha + y_0 \cdot \cos \alpha \end{cases} \quad (3-1)$$

将式 (3-1) 写成矩阵的形式为：

$$\begin{bmatrix} x_1 \\ y_1 \end{bmatrix} = \vec{P} = R_\alpha \vec{P}_1 = \begin{bmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{bmatrix} \cdot \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} \quad (3-2)$$

称式 (3-2) 中的矩阵 R_α 为二维笛卡尔坐标系中的向量旋转矩阵，简称旋转矩阵。

依据二维的旋转矩阵可以得到四旋翼飞行器的三维旋转矩阵。首先依次定义 Roll、Pitch、Yaw 三个角度的正负：绕 X 轴旋转时 Y 轴向 Z 轴靠近为正方向、绕 Y 轴旋转时 Z 轴向 X 轴靠近为正方向、绕 Z 轴旋转时 X 轴向 Y 轴靠近为正方向。按照这个定义可以求出各个轴的旋转矩阵：

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix} \quad (3-3)$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \quad (3-4)$$

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (3-5)$$

定义旋转顺序为 XYZ，从而得到 E 系向 B 系的旋转矩阵 R：

$$R = R_z(\psi) \cdot R_y(\theta) \cdot R_x(\phi)$$

$$= \begin{bmatrix} \cos \theta \cdot \cos \psi & \sin \phi \cdot \sin \theta \cdot \cos \psi - \cos \phi \cdot \sin \psi & \sin \phi \cdot \sin \psi + \cos \phi \cdot \sin \theta \cdot \cos \psi \\ \cos \theta \cdot \sin \psi & \cos \phi \cdot \cos \psi - \sin \phi \cdot \sin \theta \cdot \sin \psi & \cos \phi \cdot \sin \theta \cdot \sin \psi - \sin \phi \cdot \cos \psi \\ -\sin \theta & \sin \phi \cdot \cos \theta & \cos \phi \cdot \cos \theta \end{bmatrix} \quad (3-6)$$

通过式 (3-6) 得到了旋转矩阵 R 就可以完成 E 系和 B 系之间的物理量转换了。实际上这是用方向余弦的形式去进行姿态解算，在这过程中存在万向节死锁现象，可以通过四元数等方法消除这种情况，由于在本文所探讨的条件下不存在万向节死锁，所以就以这种方式来进行姿态解算。

3.1.2 四旋翼飞行器动力学模型

利用姿态解算的工具，结合物理规律和一些数学方法可以建立四旋翼飞行器的简化模型^[2,17]。

首先提出简化模型的假设前提：

1. 四旋翼飞行器视作一个刚体，忽略一切形变和弹性振动；
2. 四旋翼的重心就在其几何中心处，机体均匀对称于几何中心；

3. 忽略外部干扰、旋翼旋转气流的干扰，旋翼推力视为仅和转速有关物理量；
4. 空气阻力恒定。

首先是电机模型，推导过程省略，基本结论是转速的平方和推力成正比，用推力系数 k_T 表示比例关系即：

$$T = k_T \omega^2 \quad (3-7)$$

对于四旋翼飞行器的动力学模型，简化后，其动力学模型满足牛顿-欧拉方程：

$$\begin{cases} m\ddot{\vec{a}} = R\vec{T} - \vec{G} - k_{air}\vec{v} \\ \vec{M} = I\dot{\vec{\omega}} + \vec{\omega} \times I\vec{\omega} \end{cases} \quad (3-8)$$

其中， m 为模型质量， a 为加速度， R 为旋转矩阵， T 为 B 系下推力， G 为重力， k_{air} 为空气阻力， v 为速度， M 为力矩， I 为转动惯量， ω 为角速度。关于牛顿-欧拉方程这里不再推导。将四个旋翼产生的力和力矩带入上述方程，可以得到：

$$\begin{cases} \ddot{x} = (\sin \theta \cos \phi \cos \psi + \sin \phi \sin \psi) \cdot U_1/m - k_x \dot{x} \\ \ddot{y} = (\sin \theta \cos \phi \sin \psi - \sin \phi \cos \psi) \cdot U_1/m - k_y \dot{y} \\ \ddot{z} = \cos \theta \cos \phi \cdot U_1/m - g - k_z \dot{z} \end{cases} \quad (3-9)$$

以及

$$\begin{cases} \dot{\omega}_1 = [lU_2 + (I_y - I_z)\omega_2\omega_3 - I_{motor}\omega_2\Omega]/I_x \\ \dot{\omega}_2 = [lU_3 + (I_z - I_x)\omega_1\omega_3 + I_{motor}\omega_1\Omega]/I_y \\ \dot{\omega}_3 = [U_4 + (I_x - I_y)\omega_2\omega_3]/I_z \end{cases} \quad (3-10)$$

其中

$$\begin{cases} U_1 = \sum_{i=1}^4 F_i \\ U_2 = F_1 - F_3 \\ U_3 = F_2 - F_4 \\ U_4 = k_a(\Omega_1^2 + \Omega_3^2 - \Omega_2^2 - \Omega_4^2) \end{cases} \quad (3-11)$$

并且

$$\Omega = \Omega_2 + \Omega_4 - \Omega_1 - \Omega_3 \quad (3-12)$$

以上三组方程 (3-9)、(3-10)、(3-11) 和一个公式 (3-12) 组成了简化的四旋翼飞行器模型动力学描述。其中第一组式 (3-9) 为 E 系加速度方程，第二组式 (3-10) 为 B 系角加速度方程，第三组式 (3-11) 为控制量方程，附加一个对角速度变量

的解释式 (3-12)。m 为四旋翼质量， k_x 、 k_y 、 k_z 分别代表沿 E 系三个轴向的空气阻力系数（可视为相等）， k_a 表示空气阻力矩比例常数，l 为四旋翼重心（几何中心）到旋翼重心（几何中心）的距离， I_x 、 I_y 、 I_z 、 I_{motor} 分别表示 B 系沿 x、y、z 轴轴向的转动惯量以及电机旋翼的转动惯量， Ω_i 表示第 i 个电机旋翼的旋转角速度， F_i 表示第 i 个电机旋翼产生的推力。

通过建立的模型可以看到，使用四个控制量可以控制四旋翼飞行器的飞行。

3.2 控制算法

合适的控制算法是四旋翼飞行器稳定飞行的保障，目前常用的控制算法主要有传统 PID 算法、最优控制、滑模控制、反演控制等，还有模糊控制、预测控制等也开始用于四旋翼飞行器的控制并不断改进发展。由于本次课题的主要目的不是研究飞行器的飞行控制系统，所以不对其进行深入分析，在模型建立的过程中将仿照当前比较经典的内外环双闭环 PID 控制方式，内环为姿态控制器，外环为位置控制器，两个控制器之间通过姿态角相关变量进行连通。不同的是，在位置控制器中，不同于以往的控制器直接由位置信息通过 PID 控制器得到加速度信息，这里将使用 Sigmoid 函数得到期望加速度，而后对加速度的差值进行 PID 控制。为了表示方便，定义函数 PID 如下：

$$PID(E_x) = k_p \cdot E_x + k_I \int E_x dt + k_D \cdot \dot{E}_x \quad (3-13)$$

其中 k_p 、 k_I 、 k_D 分别是比例系数、积分系数和微分系数。

3.2.1 位置控制

位置控制是指针对四旋翼飞行器的位置坐标信息，求出当前线加速度、角加速度的期望值，以便后续生成合理的控制量作用于飞行器模型。该控制器的输入是四旋翼飞行器当前坐标值和期望坐标值，输出是控制量 U_1 和姿态角的期望值。控制器内部采取两级运算，首先由当前位置坐标和期望位置坐标结合控制算法得到线加速度预期值，该级运算器称为 Pos2Acc；然后根据 Pos2Acc 运算得出的线加速度期望值，通过罗德里格旋转公式（Rodrigues' rotation formula）计算出期望姿态角，该级运算器称为 Acc2Angle。

在 Pos2Acc 中，通过 Sigmoid 函数编写的运算单元，得到初始的线速度和线加速度期望值，记

$$(v_e, a_e) = \text{Sigmoid}(P, P_t) \quad (3-14)$$

对式 (3-14) 仅保留线速度，则将

$$v_e = \text{Sigmoid}(P, P_t) \quad (3-15)$$

表示为 Sigmoid 运算单元，表示通过当前位置 P_t 和期望位置 P 得到期望的速度和加速度。仅保留线速度，则 Pos2Acc 可以表示为

$$\begin{cases} E_{v_x} = \text{Sigmoid}(X, X_t) - \dot{X}_t \\ E_{v_y} = \text{Sigmoid}(Y, Y_t) - \dot{Y}_t \\ E_{v_z} = \text{Sigmoid}(Z, Z_t) - \dot{Z}_t \end{cases} \quad (3-16)$$

而后

$$\begin{cases} V_{x_e} = \text{PID}(E_{v_x}) \\ V_{y_e} = \text{PID}(E_{v_y}) \\ V_{z_e} = \text{PID}(E_{v_z}) \end{cases} \quad (3-17)$$

在 Acc2Angle 控制器中，引入罗德里格旋转公式，该公式的作用是已知某一向量和旋转后的向量反解出旋转矩阵，在此不做推导和证明。由期望加速度和模型受力情况，通过罗德里格旋转公式可以求出期望的姿态角以及控制量 U_1 ，关于罗德里格旋转公式这里不再赘述。上述过程可以表示为

$$(U_1, \Omega) = \text{Rod}(A) \quad (3-18)$$

其中

$$\Omega = \begin{pmatrix} \phi \\ \theta \\ \psi \end{pmatrix}, A = \begin{pmatrix} A_{x_e} \\ A_{y_e} \\ A_{z_e} \end{pmatrix} = \begin{pmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{pmatrix} \quad (3-19)$$

位置控制的系统示意图如图 3-3 所示。

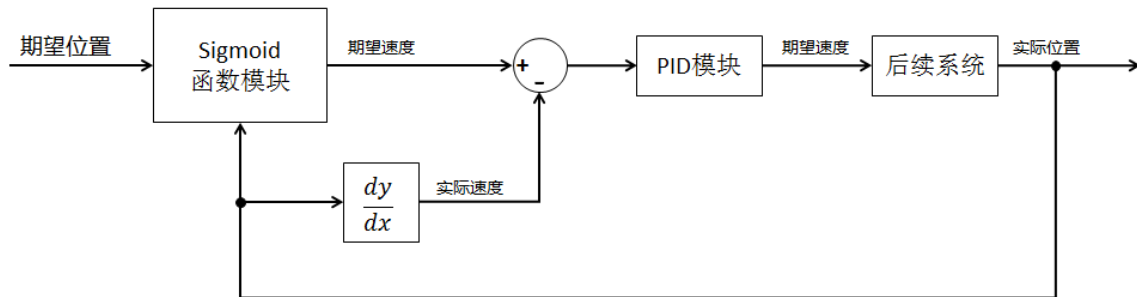


图 3-3 位置控制器系统示意图

3.2.2 姿态控制

姿态控制是指通过预期的线加速度求出四旋翼飞行器的期望角速度和角加速度。该控制器主要通过对实际值和期望值的差值通过 PID 处理而实现。因为欧拉角的存在，需要进行欧拉角的一阶导数值向机体角速度转换的处理，

$$\begin{bmatrix} p_e \\ q_e \\ r_e \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \dot{\psi}_e \end{bmatrix} + R_z \begin{bmatrix} 0 \\ \dot{\theta}_e \\ 0 \end{bmatrix} + R_z R_y \begin{bmatrix} \dot{\phi}_e \\ 0 \\ 0 \end{bmatrix}, \quad \begin{bmatrix} p_e \\ q_e \\ r_e \end{bmatrix} = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix} \quad (3-20)$$

将这个过程表示为

$$(p_e, q_e, r_e) = Euler2Body(\dot{\phi}_e, \dot{\theta}_e, \dot{\psi}_e) \quad (3-21)$$

则控制器可以表示为

$$\begin{cases} \int \dot{\phi}_e dt = PID(\phi_{e1} - \phi) \\ \int \dot{\theta}_e dt = PID(\theta_{e1} - \theta) \\ \int \dot{\psi}_e dt = PID(\psi_{e1} - \psi) \end{cases} \quad (3-22)$$

$$(p_e, q_e, r_e) = Euler2Body(\dot{\phi}_e, \dot{\theta}_e, \dot{\psi}_e) \quad (3-23)$$

$$\begin{cases} \dot{p}_e = PID(\dot{p}_e - \dot{p}) \\ \dot{q}_e = PID(\dot{q}_e - \dot{q}) \\ \dot{r}_e = PID(\dot{r}_e - \dot{r}) \end{cases} \quad (3-24)$$

3.2.3 Sigmoid 控制器

正如前面对控制算法的简单介绍中所说，在模型建立的过程中仿照了内外环双闭环 PID 控制方式，内环为姿态控制器，外环为位置控制器，两个控制器之间通过姿态角相关变量进行连通。在常规的位置控制器中，直接由位置信息通过 PID 控制器得到加速度信息，但是由于这种调节方式没有合理的物理理论基础，其 PID 参数调节需要依靠经验调节法并进行反复试验。我们知道对于四旋翼无人机来说，直接的控制量是电机的转速，其反映的是电机提供的推力。在经典力学下，由牛顿第二运动定律可知，最直观的映射力的情况的运动状态量是加速度，因此这里使用 Sigmoid 函数得到期望加速度，而后对加速度的差值进行 PID 控制。使用这种控制策略代替原来的控制方式，本质上为控制策略建立了更加合理的理论基础。具体的 Sigmoid 运算器算法如下文所述。

采用的 Sigmoid 函数为：

$$S(t) = \frac{K}{1 + e^{-a(t-t_0)}} \quad (3-25)$$

用实际意义对式 (3-25) 中参数进行解释, 则函数值就是当前位置 (位移), K 值表示的是路径总长度, a 表示的是加速度趋势, a 越大表示加速度变化越集中且越快, t_0 是速度刚达到最大值时的时间, 如图 3-4 所示。

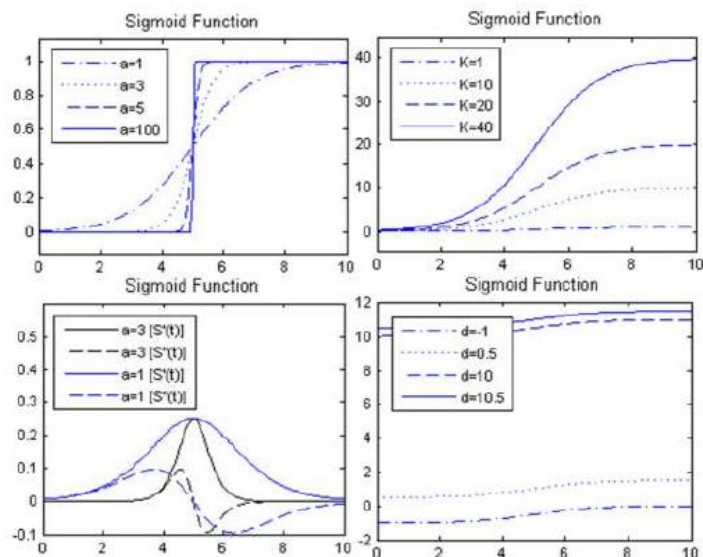


图 3-4 $t_0 = 5$ 时不同参数下的 Sigmoid 函数和导函数图像

在使用该函数时, 以函数本身为位置函数, 其一阶导函数为速度函数, 二阶导函数为加速度函数。

因为 (显然 K 不为 0)

$$S(0) = K/1 + e^{at_0} \neq 0 \quad (3-26)$$

所以需要定义最大误差值 $E_{\max}$, 满足

$$S(0) = K/1 + e^{at_0} = E_{\max} \quad (3-27)$$

由式 (3-27) 可以得到 t_0 。若 $S(t) < E_{\max}$, 就认为 $S(t)$ 为 0。

受限于实际四旋翼无人机的机械结构、电源功率、电机功率等因素, 需要设定飞行器所能达到的最大速度 $V_{\max}$ 和最大加速度 $A_{\max}$, 而速度和加速度的期望是由 Sigmoid 函数的相应导函数确定的, 易得

$$V_{\max} = \max[\dot{S}(t)] = \dot{S}(t_0) = aK/4 \quad (3-28)$$

$$A_{\max} = \pm 0.096224a^2K \quad (3-29)$$

联立以上两个方程可以得到 a 和 K 两个参数。

得到了完整的 Sigmoid 函数之后, 根据输入的位置值, 得到对应的 “时间”,

代入一二阶导函数中，由此计算当下期望的速度和加速度。就可以以由该控制器算出的期望速度和加速度进行控制。使用该控制器，可以从位移、速度和加速度三个动力学参量上对四旋翼飞行器进行追踪分析。

3.3 模型控制

通过位置和姿态控制，结合四旋翼飞行器的动力学模型，可以设定控制量 $U_{1\sim 4}$ ，将控制量 $U_{1\sim 4}$ 输入四旋翼飞行器模型，通过积分可以得到四旋翼飞行器的坐标和姿态，从而进行反馈得到闭环。按照这种方法，以 Sigmoid 函数为核心的内环姿态控制、外环位置控制的双闭环 PID 控制方式就实现了。

3.4 本章小结

本章主要介绍了四旋翼飞行器的动力学模型搭建和模型控制算法，虽然飞行控制算法对于四旋翼飞行器来说至关重要，并且对路径及运动规划有着重要影响，但是由于模型的搭建和飞行控制算法研究并不是本次课题的主要研究目标，因此在论文中仅作简要介绍。四旋翼飞行器模型的搭建主要依靠牛顿-欧拉方程，从线运动和角运动两个方面描述飞行器的运动规律。控制律采用经典 PID 控制和 Sigmoid 函数控制器相结合，沿用了常用的双闭环结构，从线运动和角运动两个方面对模型进行控制。

第四章 设计应用

4.1 路径规划条件分析

在第二章中谈到，路径规划的任务可以简单的归结为是找到在一定限制条件下从起点 A 到终点 B 的路径，有的时候因为更多的需求或任务，还会对路径规划有更多的要求和限制。因此对路径规划的条件进行分析是选择路径规划算法的必要准备工作。

本次课题提出了三种路径规划任务会遇到的情况加以分析。首先是两种全知环境下对威胁点的规避算法，针对不同的威胁点——障碍物和雷达——分别使用了不同的算法，A*算法和 Dijkstra 算法，但都是基于 Voronoi 图的路径信息^[18]；然后是对路径上的转向问题使用 Dubins 曲线加以优化。

4.1.1 威胁点的性质及筛选

本文提到的路径规划的环境要素主要是威胁点，这也是在实际应用中经常要遇到的问题。对于不同类型和性质的威胁点，相应的路径规划算法也要做出调整和修正。

作为最常见的两种威胁点，障碍物和雷达有完全不一样的性质，对路径规划的影响很大。在本次课题中，将障碍物假设为圆柱体以简化分析，对雷达也将进行简化，而不做深入的分析。障碍物和雷达最大的区别在于，障碍物的威胁等级为两点分布，即 0 或 1，或者说是有威胁和无威胁两种状态；而雷达的威胁和距离雷达的距离以及暴露在雷达有效探测半径内的时长有关，一般其威胁等级和路径的选择之间有着连续非线性关系。此外，障碍物的威胁是不存在累积性和累加性的，而雷达的威胁一方面随着暴露在雷达有效探测范围内的时间而增加，另一方面四旋翼飞行器所在位置如果在多个雷达的有效探测半径之内，则被探测到的威胁是会叠加的。接下来将对两种威胁性质进行具体的分析。

对于障碍物，经简化为圆柱体后，第 i 个障碍物的参数将由圆面半径 r_i 和障碍物高度 h_i 组成。在三维空间中，假设飞行器定高飞行，即飞行高度恒定不变，显而易见的是，若障碍物高度低于飞行器飞行高度，则不需要将其视为威胁，所以在构建 Voronoi 图之前，需要对威胁点进行筛选，对于那些所代表的障碍物高度不足以威胁四旋翼飞行器飞行的，应当剔除出威胁点集，以便减轻 Voronoi 图的计算量。路径权值的计算可以按照以下原则：

$$T_{ij} = \alpha \cdot l_{ij} \quad (\alpha = 1 \text{ or } \infty) \quad (4-1)$$

其中等式左边表示从节点 i 到节点 j 的路径权值, l_{ij} 表示路径长度, 可以通过两点之间距离公式求出, α 在该路径被障碍物中断时为无穷大, 否则为 1。判断路径是否被中断的方法就是比较威胁点到路径的距离是否大于障碍物半径, 若大于则不会被中断, 否则会被中断。

对于威胁点代表的是雷达的情况, 要复杂的多, 因为四旋翼飞行器被雷达探测到的概率计算和众多因素有关, 除了雷达本身的侦查水平外, 飞行器和雷达的距离、飞行器的飞行速度、飞行高度等通常都对飞行器是否会被雷达侦测到有很大的影响。雷达的探测机制不是本文研究的核心内容, 所以对雷达进行简化, 认为所有雷达的探测能力一样, 探测半径无限大, 飞行器被雷达探测到的概率只取决于飞行器和雷达的最近距离, 并且单个雷达的威胁程度与飞行器和雷达的最近距离的四次方成反比^[18], 则飞行器在从路径点 i 到下一路径点 j 的路径代价可以由下式计算:

$$T_{ij} = \sum_n \frac{1}{(d_n)^4} \quad (4-2)$$

其中 n 为威胁点总数。

4.1.2 算法匹配

A*算法由于使用了启发性搜索条件, 所以搜索速度在同等情况下一般快于遍历式搜索的 Dijkstra 算法。A*算法通过启发性函数可以尽量少的扩展节点, 理想情况下只会对最优路径上的节点以及其临近节点进行扩展和计算; 而 Dijkstra 算法会对所有节点进行扩展和计算。虽然 Dijkstra 算法可以得到所有节点相对源点的最小代价路径, 也就是说能够获得地图的全面信息, 但是在要求快速获得最短路径的场合, 其计算速度远不如 A*算法。然而也正是因为 A*算法使用了启发性函数加快搜索速度, 在环境复杂的场合, 路径的选取标准不单单是路径距离的远近时, A*算法往往难以找到合适的启发性条件, 或启发性函数会过于复杂, 使其失去计算速度上的优势。

对威胁点是障碍物的情形, 最优路径就是不会发生碰撞的最短路径, 其代价就是路径长度 (或者更准确的说是飞行器能耗), 此时采用 A*算法就可以很好的得到最优路径; 对威胁点是雷达的情况, 由于威胁等价或是代价的计算比较复杂, A*算法中的启发性函数难以确定, 此时采用 Dijkstra 算法可以更清晰和方便的建立权值的计算算法。

4.2 基于沃罗诺伊图的飞行路径规划

本次课题首先选用图形学 Voronoi 图作为路径规划的基础。Voronoi 图可以依据已知的威胁点或规避点生成全局最佳路径，配合一些路径规划算法，诸如本次课题选用的 A*算法和 Dijkstra 算法，在合理的选择路径权值的情况下，可以做到十分优秀的路径规划效果。

4.2.1 沃罗诺伊图的生成

在生成 Voronoi 图之前，需要明确威胁点集，正如上一节所说，对于不同性质的威胁点，有略微差别。若威胁点为障碍物，则应该剔除高度低于飞行器飞行高度的障碍物对应的威胁点，从而降低 Voronoi 图生成的计算量；而对于威胁点是雷达的情况，则不需要进行预处理。

在已知威胁点集的情况下，生成 Voronoi 图的基本方式有两种，即上文曾简单介绍过的直接法和间接法，本文将采取间接法构造 Voronoi 图。间接法是通过 Voronoi 图的对偶图，即 Delaunay 三角网实现的，故组网是首先需要完成的。组网的方式有若干种，仅简要介绍一种 Bowyer-Watson（鲍耶-沃森）算法。该方法的步骤如下：

- 1.构造一个包含所有威胁点的超级三角形，并放入三角形链表；
- 2.插入一个威胁点，在链表中找到其外接圆包含插入点的所有三角形，称这种三角形为影响三角形，删除影响三角形的公共边，将插入点和影响三角形的所有顶点连接起来；
- 3.对新形成的所有三角形按照一定准则进行局部优化，将优化后的三角形放入链表中；
- 4.循环执行步骤 2、3 直至所有威胁点插入完毕。

第三步的优化准则具体如下：

- 1.将具有公共边的三角形合成一个四边形；
- 2.以最大空圆准则对四边形进行检查，检查是否有顶点落于圆内；
- 3.若有顶点落于圆内，则修正对角线，也就是将原四边形的对角线对调。

最大空圆准则在这里是指在一个平面四边形内，选取三个点作由这三个点构成的三角形的外接圆，检查剩余的顶点是否在该圆内，如果剩余的顶点在该圆内，则该圆不是这个四边形的最大空圆，否则反之。

在组网完成之后，计算每个三角形外接圆圆心，并记录这些圆心为路径点，遍历三角形链表，寻找与当前三角形有公共边的所有三角形，连接其外接圆圆心作为 Voronoi 边，对于孤立边，求出其中垂线射线作为 Voronoi 边。当对所有三角

形都执行过这一系列的操作之后，Voronoi 图就构造完成了。

4.2.2 沃罗诺伊图的使用

在 MATLAB 中，有附带 Voronoi 图相关功能的工具箱，也有集成的 Voronoi 函数，这里使用相对简单的 Voronoi 函数。MATLAB 中的 Voronoi 函数的生成方式是通过构建 Delauney 三角网间接得到 Voronoi 图，如 $[VX, VY] = \text{voronoi}(X, Y)$ ， X 、 Y 表示威胁点的 X 坐标集合和 Y 坐标集合， VX 、 VY 表示 Voronoi 边的顶点 X 坐标集合和 Y 坐标集合；还有 $[V, C] = \text{voronoin}(A)$ ， A 为威胁点点集， C 和 V 分别表示威胁点邻近的 voronoi 边顶点序号和顶点序号对应的坐标。通过这两个函数可以建立点集和边集一一对应的映射关系。

在实际应用时，首先使用 $[VX, VY] = \text{voronoi}(X, Y)$ 和 $[V, C] = \text{voronoin}(A)$ 得到点集和边集，由于 VX 和 VY 所表示的边的顶点的坐标会存在重复并且顺序比较混乱，所以首先需要进行路径点和路径的提取和对应。然后根据 V 和 C 得到路径和威胁点的关系，即把威胁点和其近邻的路径相关联，以便计算路径的权值。

对于威胁点是障碍物的情况，构造的 Voronoi 图如图 4-1 所示。

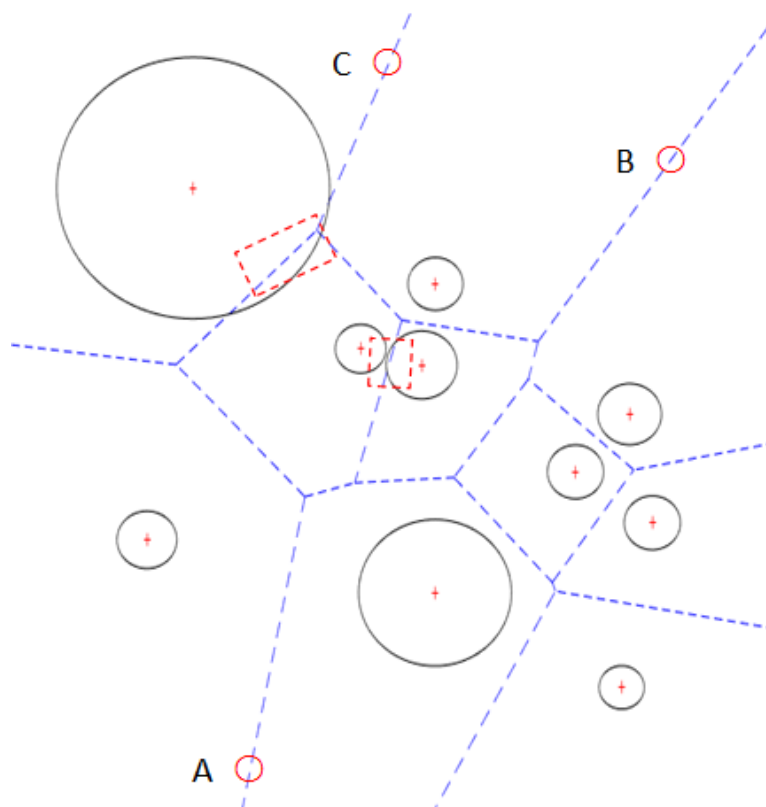


图 4-1 对障碍物威胁生成的 Voronoi 图

在图 4-1 中，随机生成了 10 个威胁点分别作为 10 个障碍物的圆心，又在合理

范围内生成了 10 个伪随机数作为障碍物的半径，这里暂且假设所有障碍物的高度均大于飞行高度，因此不因高度因素去除威胁点。构造出 Voronoi 图之后可以看到，由于蓝色虚线代表的备选路径是距离圆心最远的路径，所以成功避开了大部分障碍物，只有在两处被红色方框指示出的地方，路径受到了障碍物的中断。面对障碍物中断路径的情况，有两种解决办法，一是重新生成原来被中断的路径，使新的路径按照一定方法绕开障碍物；二是认为被中断的路径无效，不再考虑该路径。在障碍物较为稀疏的情况时，若从出发点到目标点之间的大部分路径是通路，可以放弃少数被中断的路径，以换取计算速度，如图 4-1 中从 A 点到 B 点。但是如果障碍物中断了从出发点到目标点的主要路径，如图 4-1 中从 A 点到 C 点，就必须采取其他方法生成新的路径，这也是 Voronoi 图在应用于障碍物规避时要面对的一个问题。另外，正如前文提到的，图中可以看到有大量的空间被浪费，比如从 A 到 B 明显有更近的路线，这也是这种情况下使用 Voronoi 图最大的缺点。

对于威胁点是雷达的情况，则不需要考虑路径被遮挡的情况，直接以所有雷达位置为威胁点画出 Voronoi 图即可。

4.2.3 A*算法实现障碍物规避

A*算法是启发式搜索算法的典型代表，同时也是求取最短路径的典型算法，合理的设置启发式函数可以极大增加搜索速度，缩短搜索时间。

由于障碍物规避问题中，不存在复杂的环境，只是求取备选路径中最短的一条，所以 A*算法十分适用于这种情况。一般来说，对于普通的最短路径规划问题，可以直接定义启发式函数为当前节点到目标点的直线距离，这种定义方法不仅容易计算，并且满足式（2-4）即估计的路径长度不大于实际路径长度，根据 Peter E. Hart 等人的研究结论，满足这个公式的启发式函数能够保证得到的路径是最短路径。

通常 A*算法是用来直接求取最短路径的，所以路径的权值就是路径的长度，换句话说，四旋翼无人机在路径上的代价为燃油/耗能代价，对于这种情况可以表示为

$$T = \begin{cases} \infty, & r \geq kd \\ 1, & r < kd \end{cases} \quad (4-3)$$

其中 T 为代价系数，r 为障碍物中心到边缘的距离，d 为飞行器距离障碍物中心的距离，k 为安全系数且小于 1。

在仿真时，随机生成了 20 个威胁点，在这里假设障碍物足够稀疏且半径差异不大，预先生成的 Voronoi 图中的路径不存在被中断的情况。算法生成的结果如图

4-2 所示。其中红色加号为威胁点，蓝色虚线为 Voronoi 备选路径，红色实线为最短路径。

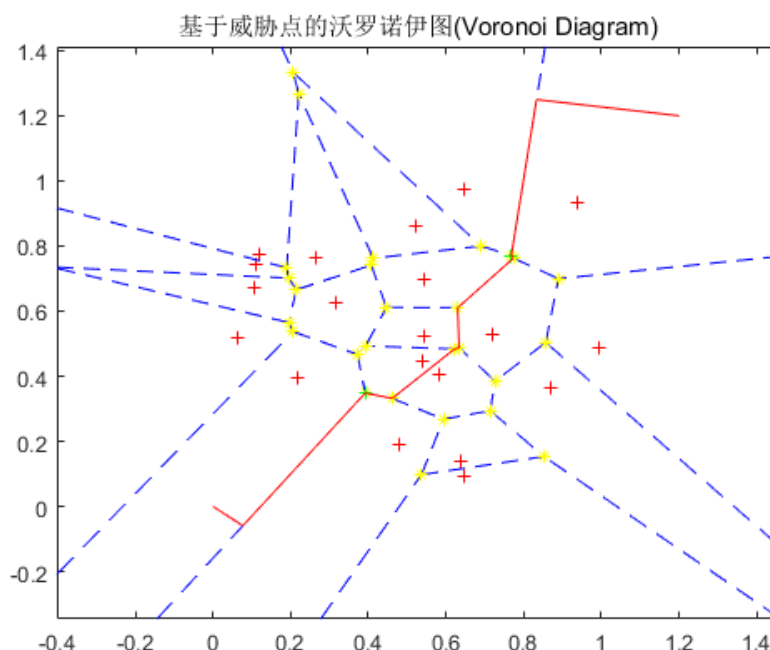


图 4-2 A*算法实现障碍物规避仿真

4.2.4 迪杰斯特拉算法实现雷达规避

迪杰斯特拉算法是一种经典的有向图最短路算法，该算法由路径点坐标和路径权值为输入，输出当前权值下从源点到所有其他路径点的最短路径^[19]。

假设路径点集 P 中有 18 个点 $P_{1\sim 18}$ 并表示为

$$P = \begin{pmatrix} x_1 & y_1 \\ \vdots & \vdots \\ x_{18} & y_{18} \end{pmatrix}$$

点集 P 的连接关系表示在路径边集 L 中，假设共有 20 条路径

$$L = \begin{pmatrix} l_{1,1} & l_{1,2} & l_{1,3} \\ \vdots & \vdots & \vdots \\ l_{20,1} & l_{20,2} & l_{20,3} \end{pmatrix}$$

其中每一行代表一组路径信息，每组路径信息的第一个元素表示路径起始点序号，第二个元素代表路径终止点序号，第三个元素代表该路径权值。例如 L 中的某一行 1,2,3，就代表这条路径是从点集 P 中的 (x_1, y_1) 指向 (x_2, y_2) ，且权值为 3。

将 L 转换为邻接矩阵 X ，在这个过程中，有一个很重要的改变；由于 Dijkstra

算法仅对有向图有效，而 L 中的路径实际是不区分方向的，所有如果将 L 直接变为邻接矩阵，会出现死循环或者点遍历不全的情况，因此需要将 X 扩展为对角线对称矩阵，即 X 中的第 i 行第 j 列和第 j 行第 i 列元素相等。本次课题使用的实际的 Dijkstra 算法就是对邻接矩阵 X 进行处理而得到的最短路径。

在展示用 MATLAB 软件编写的路径规划成果之前，这一小节将会对此前提到的一个问题进行补充说明，即代价的量化。

因为 Dijkstra 算法是对带权路径进行最短路规划的（如果没有说明默认权值为 1），所以在应用算法之前需要对求出的预选路径的权值进行量化计算，这也可以看做是对路径代价的量化。正如前面提到过的，在四旋翼飞行器乃至各种飞行器实际的飞行过程中，可以考虑的影响因素/代价有很多，包括但不限于环境因素、飞行器自身因素、威胁类型甚至人为因素；在仿真的过程中，选取其中比较典型的因素用以生成权值。

首先是燃油代价，即飞行距离和燃油消耗呈现正相关，简化考虑可认为是成正比的，可以表示为

$$t_1 = k_1 l \quad (4-4)$$

其中 t 为代价权值， k_1 为距离到代价的换算系数， l 为路径长度。

然后是威胁代价，大体上威胁等级和飞行器距离威胁点（雷达、哨所等）的距离呈现负相关，对于这种间接威胁，相较障碍物威胁，表达式会复杂的多，此处以一种简化的计算方法为例，认为此类威胁等级和距离的四次方成反比

$$t_2 = k_2 / d^4 \quad (4-5)$$

其中 k_2 是飞行器距威胁点距离到代价的换算系数。

将式（4-4）和式（4-5）对应的两种威胁按照加权法计算出路径权值

$$t = at_1 + (1 - a)t_2 \quad (4-6)$$

得到了路径的最终权值。

结合以上提到的 Voronoi 图、Dijkstra 算法、路径权值三个部分，并对整体程序进行一定的优化，通过 MATLAB 可以计算出符合所规定代价计算方式的相对最佳路径，运行结果如图 4-3 所示。和 A*算法的仿真图类似，图 4-3 中红色折线就是最优路径，红色“+”号是威胁点，黄色节点是路径点，蓝色虚线是 Voronoi 图，也可以称为备选路径。可以直观的看到，路径的选择基本避开了威胁点，并且从区域内选择了较为近的路线。

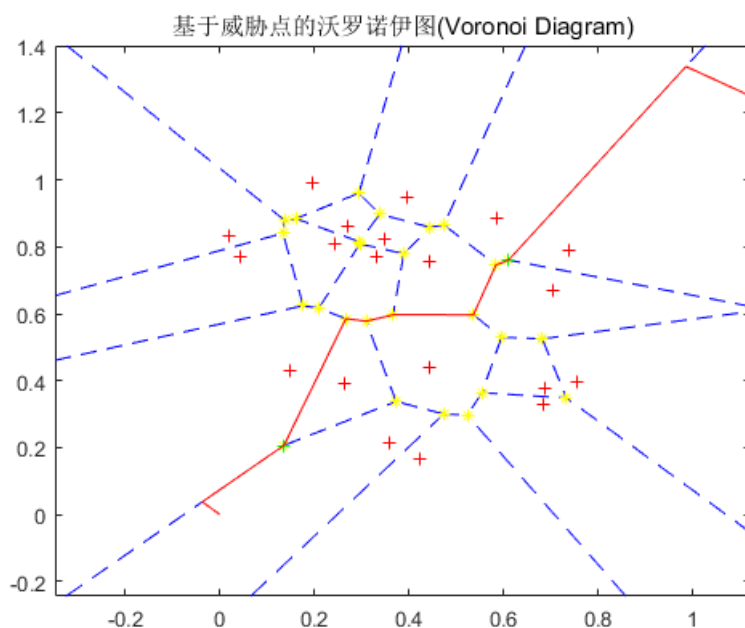


图 4-3 Dijkstra 算法最短路径规划仿真

虽然路径的规划是在二维平面完成的,但是由运动的叠加性可知,竖直方向的飞行要求可以单独实现,然后叠加到二维水平运动中,从运动学的角度来说两者并不冲突。本次实验假设四旋翼飞行器为定高飞行,对三维空间的运动分析将在下一章介绍。

4.2.5 飞行轨迹设定

在经由 Voronoi 图和最短路算法求出了最佳路径之后,就可以将四旋翼飞行器的飞行过程分为一段一段的直线飞行,这样就可以方便的对每一段飞行进行规划。

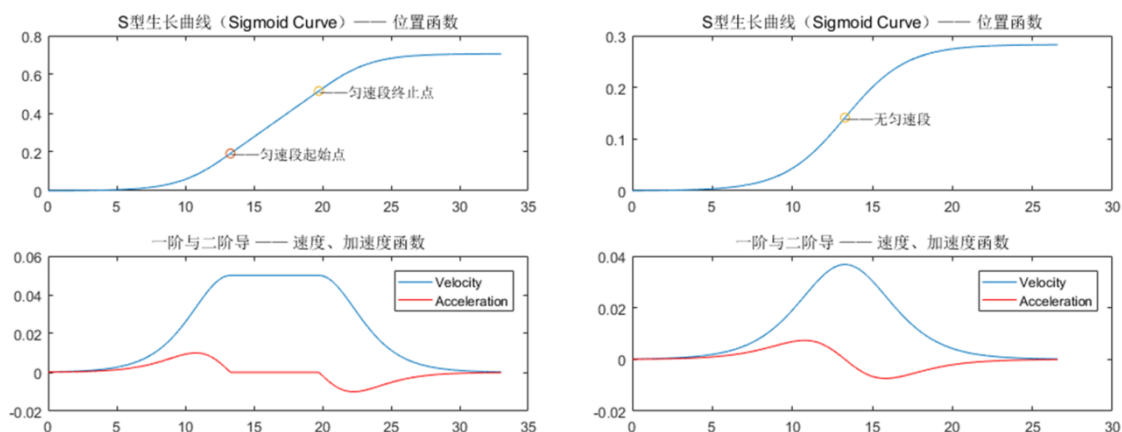


图 4-4 飞行轨迹规划

使用 Sigmoid 模块生成的理想飞行状态如图 4-4 所示。按照飞行距离的长短，飞行轨迹分为有匀速段和无匀速段两种情况。若当前路径长度过短，使得无人机无法加速到最大速度，就不存在匀速段；否则无人机将以最大速度通过匀速段。

4.3 基于杜宾斯曲线的转向策略

使用 Dubins 曲线优化转向策略时，在已知初始点和终止点、初始航向和终止航向的情况下，需要求出切圆半径和切圆圆心坐标。此处继承上文的假设，即两点足够远是的 Dubins 曲线由“曲线-线段-曲线”三段构成，对“足够远”的具体定义在上文已经说明^[20]。

切圆半径就是无人机的最大转弯半径，通常这个转弯半径和无人机的实时速度有关，可以表示为：

$$R = \frac{v^2}{g\sqrt{n_n^2-1}} = \frac{v^2}{d_0} \quad (4-7)$$

其中 d_0 为和无人机性能有关的常数，为 n_n 法向过载，与速度 v 和重力加速度 g 共同限定了最小转弯半径 R 。

对应的转弯控制量，即转弯角速度为

$$\omega = \frac{v}{R} = \frac{g\sqrt{n_n^2-1}}{v} = \frac{d_0}{v} \quad (4-8)$$

有了切向圆的半径，根据几何关系就可以求出圆心，由于初始点和终止点的求取方式一样，故只选取初始点进行说明。设初始点 P_s 的坐标为 (x_s, y_s) ，出发航向为 φ_s ，则切向圆圆心坐标 (x_{sc}, y_{sc}) 可以表示为

$$\begin{cases} x_{sc} = x_s + R\cos(\varphi_s \pm \frac{\pi}{2}) \\ y_{sc} = y_s + R\sin(\varphi_s \pm \frac{\pi}{2}) \end{cases} \quad (4-9)$$

式 (4-9) 中加减分别表示左圆（逆时针圆）和右圆（顺时针圆）。终止点的圆心坐标表示方法相同。

接下来选取切向圆，选取方式十分直观，对分别求取初始点左圆圆心到终止点两个圆圆心的距离，再分别求取初始点右圆圆心到终止点两个圆圆心的距离，选取这四个距离中最近的距离，该距离对应的两个圆就是选取的圆。用图的形式表示就是以两个航向所在直线相交于一点，以该点为顶点，分别到初始点以及终止点作射线，组成了一个区域，选取的圆应该在这个区域中。

在选取了圆后，称初始点处选取的圆为初始圆，终止点处选取的圆为终止圆。从初始圆到终止圆有四条公共切线如图 2-4 所示。四条切线中只有一个是符合起止航向的，关于切线的选取办法为，设初始圆上任意切点 C_i ，对向量 $\overrightarrow{P_s C_i}$ 和初始航向对应向量求内积，若结果大于 0 则表示该切点对应方向和初始航向一致，可将该切点作为候选，否则舍弃这个切点。对终止圆做类似的操作，唯一不同的一点是向量选取，应该从切点向终止点构造向量即 $\overrightarrow{C_i P_e}$ 。最终可以在两个圆上各得到两个候选切点，其中只有两个是在同一条切线上的，这条切线就是我们选取的 Dubins 曲线的直线段部分。自此 Dubins 曲线就完全生成了。

将 Dubins 曲线实际应用于原始路径的优化时会产生一个问题，即原始路径是由折线组成的，Dubins 曲线规划出的路径无法和原始路径重合，如图 4-5 所示。

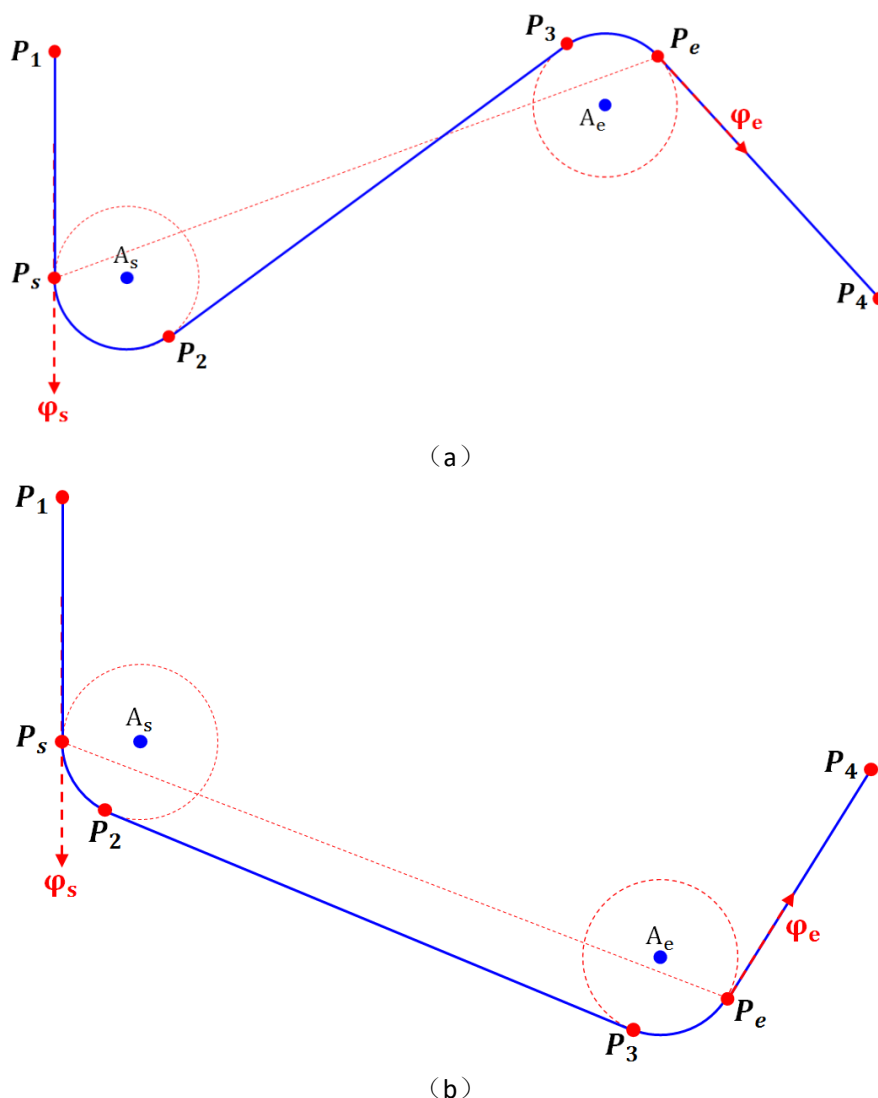


图 4-5 对原路径修正的 Dubins 曲线路径。(a) 线段部分为内切线；(b) 线段部分为外切线

在图 4-5 中, 原始路径都是 $P_1 \rightarrow P_s \rightarrow P_e \rightarrow P_4$, 应用 Dubins 曲线后的路径是 $P_1 \rightarrow P_s \rightarrow P_2 \rightarrow P_3 \rightarrow P_e \rightarrow P_4$, 也就是将原本从 $P_s \rightarrow P_e$ 的直线段路径替换为由圆弧-线段-圆弧组成的 Dubins 曲线, 图中的虚线圆是帮助构成 Dubins 曲线的辅助切向圆。显而易见的是, 修正了转向策略的路径和原路径并不重合, 无论是选取内切线还是外切线为直线段路径, 和原路径必定无法重合, 这是因为为了保持一定飞行速度而增加的圆弧路径带来的转向延迟所导致的。对于路径偏离的现象有两种不同情况, 一是辅助切向圆的半径远远小于障碍物之间的距离, 因为路径的偏离距离不大于辅助切向圆的直径, 并且更多时候是小于其半径的, 所以此时可以认为路径的偏离是可以容忍的, 或者说是允许的, 安全的; 二是当这种偏离相对障碍物之间的距离来说不可忽略时, 需要考虑对原算法进行修正, 修正策略是将转向提前, 保证 Dubins 曲线的线段部分和原路径可以重合, 对图 4-5 (a) 所示的 Dubins 路径的修正如图 4-6 所示, 新的路径为 $P_1 \rightarrow P_{s1} \rightarrow P_2 \rightarrow P_3 \rightarrow P_{e1} \rightarrow P_4$, 即在 P_{s1} 和 P_{e1} 两处提前转向, 保证修正后的路径直线段和原路径重合。

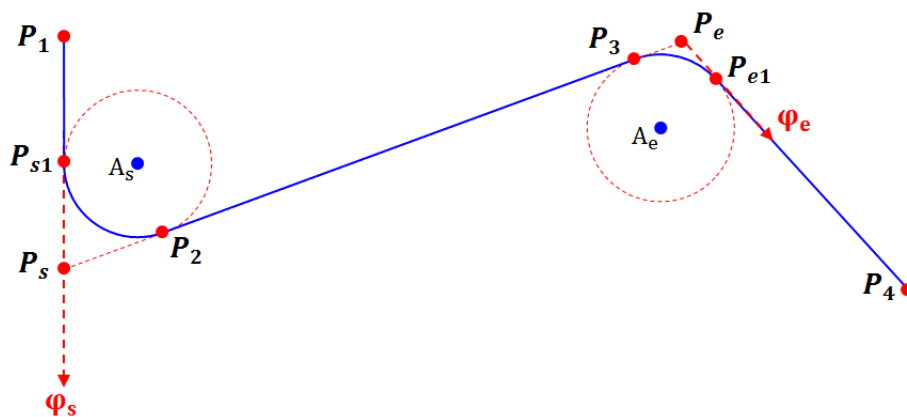


图 4-6 Dubins 曲线的修正

为了实现对 Dubins 曲线的修正, 需要选择新的转向点, 实际上, 这项修正会使算法的生成更为简单, 只需要以最小转弯半径在路径的拐点处作拐角的切线圆, 即和两条路径都相切的圆, 切点就是新的转向点。对于图 4-5 (b) 的情况修正方法也一样, 这里不再赘述。

在生成了 Dubins 曲线之后, 此前使用 Sigmoid 函数进行的轨迹规划就无效了。由于原先是由直线段组成路径, Sigmoid 函数规划按照每一段路径进行规划, 每一段的起点和终点就是路径点的折点, 如图 4-6 中的 P_s 和 P_e 两个点, 并且在两个点的速度均为 0, 在每一段路径的中间速度达到改段路径的最大值; 修正后, 以刚刚离开圆弧段为起点 (如图 4-5 中的 P_2 、 P_e , 图 4-6 中的 P_2 、 P_{e1}), 即将进入圆弧段为

终点（如图 4-5 中的 P_s 、 P_3 ，图 4-6 中的 P_{s1} 、 P_3 ），要求在新的起点和终点处速度为最小速度（速度和切向圆半径的关系见式 4-7），这种情况下不能直接使用 Sigmoid 函数进行轨迹规划，在速度和加速度的选择上需要进行一定的更改。

4.4 本章小结

本章主要介绍了以第一章叙述的研究内容为目标，第二章叙述的相关理论为基础，第三章叙述的四旋翼飞行器动力学模型及控制模型为参考形成的研究成果，包括基于威胁点的路径规划、飞行轨迹设定和转向策略的研究，依次涉及到 Voronoi 图、A*算法和 Dijkstra 算法，Sigmoid 函数，Dubins 曲线。

本次课题的主要研究成果在本章呈现，其中以基于威胁点的路径规划为主体，轨迹规划和转向策略为对路径规划的补充，对完整运动规划过程的完善，总体上提出了一种应用于四旋翼飞行器自动路径规划及自主飞行的算法流程，对于本章内容的部分分析将在下一章提到并总结。

第五章 分析和总结

5.1 设计流程

论文的展开点为四旋翼飞行器的路径规划问题，全文从基于威胁点的路径规划和四旋翼无人机的建模及轨迹规划两方面入手。关于基于威胁点的路径规划设计思路如图 5-1 所示。



图 5-1 基于威胁点的路径规划设计思路

5.2 可行性分析

对第三章建立的四旋翼飞行器模型进行分析，由式（3-9）到式（3-12）可以得到图 5-2 所示关系。

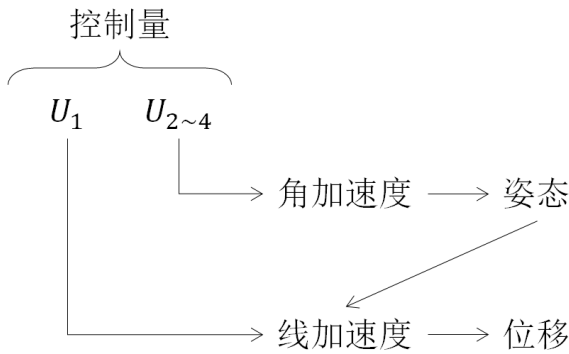


图 5-2 四旋翼飞行器的模型分析

由图 5-2 可以看到，四个控制量是通过直接改变四旋翼飞行器的角加速度，进而改变飞行器的姿态，从而影响线加速度，最终使四旋翼飞行器发生所期望的位移。四个控制量是四个旋翼产生的推力，或者说是四个电机的转速，一般认为电机可以由一阶模型近似表示。

在稳定性上，控制信号的延迟可以忽略不计，主要影响稳定性的因素除了环境外，是电机的惯性环节和飞行器刚体模型的惯性环节，然而，和固定翼飞行器不同的是，四旋翼飞行器可以通过姿态的调整对位移进行调整，这种调整是双向的，即既可以加大位移，也可以减小位移，这是固定翼飞行器所不能做到的。也正是四旋翼飞行器的这种特性，使得由 Voronoi 图构造的折线路径可以直接使用，而不需要进行转弯处的曲线化。不过，为了保证一定的速度，比如在雷达威胁下应该尽快通过雷达监测区域，则需要设定一种更为优化的转向策略，本文采用的是 Dubins 曲线。另外，从四旋翼飞行器的控制特点出发，本文借鉴了 Sigmoid 函数的扩展形式作为轨迹规划的指导函数。

5.3 总结

本次课题的研究目标是设计四旋翼飞行器的路径规划算法，本文首先简要介绍了四旋翼飞行器和路径规划问题的历史、背景和发展现状，说明了本文研究内容的大致概念和重要意义，为研究的进行起到了铺垫作用；之后详细说明了本文使用的规划方法的理论背景，从路径规划的任务入手，依次介绍了用于构建地图信息的 Voronoi 图、用于最优路径规划的 Dijkstra 算法和 A*算法、用于对路径规划结果加以优化的 Dubins 曲线、用于对每段路径飞行轨迹加以指导的 Sigmoid 函数，形成了一套完整的理论体系，保证了具体实施和验证路径规划结果的顺利进行。

在进行路径规划算法的具体实现之前，对四旋翼飞行器建立了动力学模型，以便能够对飞行器的飞行过程有一定的了解，这是验证算法是否实用的唯一途径，另外还对常用的一种双闭环控制方式进行了研究，加入了 Sigmoid 控制器的设想。通过对四旋翼飞行器进行姿态解算、物理建模、模拟控制，得到了四旋翼飞行器的位移是经由调整姿态而产生的这一重要结论，验证了四旋翼飞行器的灵活性，由于对四旋翼飞行器的建模、姿态解算以及其控制方式不是本次课题的主要研究目标，所以没有详细展开说明。

有了充分的理论基础和完备的四旋翼飞行器模型，就可以通过对通用的路径规划算法进行结合和修正得到有针对性路径规划方法。这里的针对性主要表现在以下几个方面：

1. 本文对路径规划的环境进行了分析，并针对基于威胁点规避的路径规划目标

使用 Voronoi 图建立地图信息，开展路径规划任务；

2.对威胁点是障碍物的情况，采用运算速度更快的启发式搜索算法——A*算法生成最优路径，而对威胁点是雷达或哨所等情况，采用能够兼容复杂路径权值的 Dijkstra 算法生成最优路径，针对不同的威胁点性质，采用了不同的分析方法和最优路算法，以应对可能的不同飞行环境；

3.尽管四旋翼飞行器具有可悬停转向的优势，但是针对在飞行过程中有可能出现最小速度限制的情况，对原本的折线顶点处定点转向的方法进行了优化，使用 Dubins 曲线兼容了最小速度限制的情况；

4.针对四旋翼飞行器的飞行特点，尝试使用 Sigmoid 函数作为每段路径的指导函数，对飞行过程中的位移、速度和加速度的确定均可以加以指导和辅助。

本文的主要优点和创新点也正是对不同源于以上几点。

当然，由于时间的限制和知识、人力的储备不足，以及实验设备的限制，论文中还有很多可以改进的地方，比如可以更好的结合四旋翼飞行器的飞行特点和控制特点提出更独特的路径规划方法乃至运动规划方法，还有对 Dubins 曲线优化后的轨迹没有提出明确的运动规划方法等，这也是今后需要努力的方向。

随着人工智能的发展、传感器技术的提高、控制理论的完善，在未来的路径规划之路上，更先进的、更泛化的、更智能的、更综合的路径规划算法一定会不断被提出；分布式计算、物联网技术、多智能体技术也将对飞行器的路径规划问题注入新的血液，并提出新的挑战。

致 谢

本论文的工作是在我的导师李瑞副教授的悉心指导下完成的。李老师知识渊博，待人和蔼可亲，思路清晰超前，不论是在论文的完成工作上还是对我思维的启发上都给予了极大的帮助。李老师不仅有着过硬的专业知识储备，也能把相关问题讲解的通俗易懂，极大的加速了我的研究学习速度和论文写作进展。此外，我还要感谢蒋宏学长在四旋翼飞行器的建模上给予的帮助。

参考文献

- [1] Pedro Castillo, Alejandro Dzul, Rogelio Lozano. Real-Time Stabilization and Tracking of a Four-Rotor Mini Rotorcraft[J]. IEEE Transactions of Control System Technology, 2004, 12(4): 510-516
- [2] 刘志军, 吕强, 王东来. 小型四旋翼直升机的建模与仿真控制[J]. 计算机仿真, 2010, 27(7): 18-20,69
- [3] 陈雯雯. 小型四旋翼无人机轨迹规划算法研究[D]. 青岛: 青岛理工大学, 2015
- [4] M. Bendjedja, Y. Ait-Amirat, B. Walther. DSP Implementation of Rotot Position Detection Method for Hybrid Stepping Motors[C]. Power Electronics and Motion Control Conference, 2006
- [5] J. Pan, Y. Wang, J. Wu, et al. A Trajectory Tracking Control Strategy for a Novel Multi-rotor Aircraft[C]. The 28th Research Institute of China Electronics Technology Group Corporation, Nanjing, 2016, 3273-3277
- [6] J.T.Schwartz, M.Shair. A Survey of Motion Planning and Related Geometric Algorithm[J]. Artificial Intelligence, 1998, 37(1-3): 157-169
- [7] 王庆江, 高晓光, 符小卫. 无威胁情况下任意两点间的无人机路径规划[J]. 系统工程与电子技术, 2009, 31(9): 2157-2162
- [8] G. Apostolo. The Illustrated Encyclopedia of Helicopters[M]. New York: Random House Value Publishing, 1984
- [9] 张广林, 胡小梅, 柴剑飞, 等. 路径规划算法及其应用综述[J]. 现代机械, 2011, 5: 85-90
- [10] F. Aurenhammer. Voronoi Diagrams – A Survey of a Fundamental Geometric Data Structure[J]. ACM Computer Surveys, 1991, 23(3): 345-405
- [11] 宗大伟. Voronoi 图及其应用研究[D]. 南京: 南京航空航天大学, 2006
- [12] X. K. Jiang, H. J. Fan, D. Kang. Optimization Strategy Based on Shortest Path Algorithm of Dijkstra[C]. The 2nd International Conference on Communication Technology, 2015, 78-82
- [13] P. E. Hart, N. J. Nilsson, B. Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths[J]. IEEE Transactions of Systems Science and Cybernetics, 1968, 4(2): 100-107
- [14] W. Zhu, L. Liu, L. Teng, et al. Enhanced Sparse A* Search for UAV Path Planning Using Dubins Path Estimation[C]. The 33rd Chinese Control Conference, Nanjing, 2014

- [15] L. E. Dubins. On Curves of Minimal Length with a Constraint on Average Curvature, and with Prescribed Initial and Terminal Positions and Tangents[J]. American Journal of Mathematics, 1957, 79(3): 497-516
- [16] 陈雯雯, 刘明, 雷建和, 等. 基于 Sigmoid 函数的四旋翼无人机轨迹规划算法[J]. 控制工程, 2016, 23(6): 922-927
- [17] 杨庆华, 宋召青, 时磊. 四旋翼飞行器建模、控制及仿真[J]. 海军航空工程学院学报, 2009, 24(5): 500-502
- [18] 符小卫, 高晓光. 一种无人机路径规划算法研究[J]. 系统仿真学报, 2004, 16(1): 20-21+34
- [19] C. Ruan, J. P. Luo, Y. Wu. Map Navigation System Based on Optimal Dijkstra Algorithm[C]. IEEE 3rd International Conference on Cloud Computing and Intelligence Systems, 2014
- [20] C. J. Yang, L. F. Liu, J. Wu. Path Planning Algorithm for Small UAV Based on Dubins Path[C]. International Conference on Aircraft Utility Systems, 2016

外文资料原文

A Formal Basis for the Heuristic Determination of Minimum Cost Paths

PETER E. HART, MEMBER, IEEE, NILS J. NILSSON, MEMBER, IEEE, AND BERTRAM RAPHAEL

Abstract—Although the problem of determining the minimum cost path through a graph arises naturally in a number of interesting applications, there has been no underlying theory to guide the development of efficient search procedures. Moreover, there is no adequate conceptual framework within which the various ad hoc search strategies proposed to date can be compared. This paper describes how heuristic information from the problem domain can be incorporated into a formal mathematical theory of graph searching and demonstrates an optimality property of a class of search strategies.

I. INTRODUCTION

A. The Problem of Finding Paths Through Graphs

MANY PROBLEMS of engineering and scientific importance can be related to the general problem of finding a path through a graph. Examples of such problems include routing of telephone traffic, navigation through a maze, layout of printed circuit boards, and mechanical theorem-proving and problem-solving. These problems have usually been approached in one of two ways, which we shall call the *mathematical approach* and the *heuristic approach*.

1) The mathematical approach typically deals with the properties of abstract graphs and with algorithms that prescribe an orderly examination of nodes of a graph to establish a minimum cost path. For example, Pollock and Wiebenson^[1] review several algorithms which are guaranteed to find such a path for any graph. Busacker and Saaty^[2] also discuss several algorithms, one of which uses the concept of dynamic programming.^[3] The mathematical approach is generally more concerned with the ultimate achievement of solutions than it is with the computational feasibility of the algorithms developed.

2) The heuristic approach typically uses special knowledge about the domain of the problem being represented by a graph to improve the computational efficiency of solutions to particular graph-searching problems. For example, Gelernter's^[4] program used Euclidean diagrams to direct the search for geometric proofs. Samuel^[5] and others have used ad hoc characteristics of particular games to reduce

the "look-ahead" effort in searching game trees. Procedures developed via the heuristic approach generally have not been able to guarantee that minimum cost solution paths will always be found.

This paper draws together the above two approaches by describing how information from a problem domain can be incorporated in a formal mathematical approach to a graph analysis problem. It also presents a general algorithm which prescribes how to use such information to find a minimum cost path through a graph. Finally, it proves, under mild assumptions, that this algorithm is optimal in the sense that it examines the smallest number of nodes necessary to guarantee a minimum cost solution.

The following is a typical illustration of the sort of problem to which our results are applicable. Imagine a set of cities with roads connecting certain pairs of them. Suppose we desire a technique for discovering a sequence of cities on the shortest route from a specified start to a specified goal city. Our algorithm prescribes how to use special knowledge—e.g., the knowledge that the shortest road route between any pair of cities cannot be less than the airline distance between them—in order to reduce the total number of cities that need to be considered.

First, we must make some preliminary statements and definitions about graphs and search algorithms.

... ..

外文资料译文

关于最小代价路径启发式决策的基本原理

作者: PETER E. HART, NILS J. NILSSON, BERTRAM RAPHAEL

摘要

——虽然通过图表决策最小代价路径的问题引出了很多有趣的技术手段，但是至今为止还没有一种根本的理论能够指导有效搜索过程的发展。另外，还没有一个概念框架能够让至今提出的各种特设搜索策略相互比较。本文描述了问题领域内启发式信息是如何包含于一个正式的图搜索数学理论的，并且论证了一类搜索策略的最优性质。

一、介绍

A. 通过图表找到路径的问题

很多工程和科学上的重要性问题可以被联系到通过图表寻找路径的一般问题上。这样的例子包括电话线路规划问题、迷宫导航问题、电路板布局问题以及机械原理证明和问题解决。这些问题通常是通过两种方法中的一种去处理，它们被称作数学方法和启发式方法。

1) 数学方法特别用来处理抽象图的性质问题，其使用的算法规定了对一个图的节点的有序检验，从而建立一个代价最小化路径。举个例子，Pllock 和 Wiebenson 评价了几种能够保证在任何图中找到代价最小路径的算法。Busacker 和 Saaty 也讨论了几种算法，其中有一种算法使用了动态规划的概念。相比于考虑已发展的算法的计算可行性，数学手段总体上更偏向于和达成解决方案的极限实现相联系。

2) 启发式方法的特点是使用了从问题域中得到的由图展现出的专门的经验知识，以对特定的图搜索问题提升解决方案的计算效率。举个例子，Gelernter 的课题使用了欧几里得图来引导几何证明的搜索。Samuel 等人使用了特定博弈问题的特殊特征来减少博弈树搜索中的预先性结果。启发式方法发展出的搜索过程基本上还没能保证总是找到最小代价路径。

通过描述在图分析问题中，问题域的信息是如何被合并到正式的数学方法中的，本文将上述提到的数学方法和启发式方法结合在一起。本文也展示了一种通用的算法，这种算法规定了如何通过这种信息去找到最小代价路径。最后，本文证明了，通过适当的猜想这种算法在检验节点的最小值是保证最小代价路径的前提这个层面上是最佳的。

接下来是对一类问题的一个典型论证，本文得到的结果对该类问题是有应用性的。设想一系列城市，其中对应的城市之间有道路相连。认为我们需要一种技术得到从特定城市出发到特定城市的最短路径上的城市序列。本文提出的算法规定了如何使用特定的经验知识，比如任何一对城市之间的最短路径不可能比城市之间的直航航线要短（可以理解为城市之间的直线距离，译者注），这种经验知识可以减少需要考虑的城市节点总数。

首先，我们需要对图和搜索算法做一些准备性的陈述和定义。

.....

（这篇论文发表在 1968 年的 IEEE 期刊上，首次提出了 A*算法，A*算法直到今天也被认为是最好的最短路径算法之一。）